CUI

Lab Manual

Parallel and Distributed Computing
CSC-334

Department of Computer Science
Islamabad Campus

Lab Contents

Distributed Systems; Parallel Computing; Virtual Machines & Virtualization; Parallel Algorithms & Patterns; OpenMP; GPU Concepts & Architectures; and GPU Programming Model.

Student Outcomes (SO)

S.#	Description
1	Identify, formulate, research literature, and solve complex computing problems reaching substantiated conclusions using fundamental principles of mathematics, computing sciences, and relevant domain disciplines
2	Design and evaluate solutions for complex computing problems, and design and evaluate systems, components, or processes that meet specified needs with appropriate consideration for public health and safety, cultural, societal, and environmental considerations
3	Create, select, adapt and apply appropriate techniques, resources, and modern computing tools to complex computing activities, with an understanding of the limitations
4	Function effectively as an individual and as a member or leader in diverse teams and in multi-disciplinary settings.

Intended Learning Outcomes

Sr.#	Description	Blooms Taxonomy Learning Level	SO
CLO -1	Implement parallel programming algorithms using OpenMP GPU.	*Applying*	2,4
CLO -2	Develop an application using concepts of distributed and parallel computing in a team environment.	*Creating*	2-5,9

Lab Assessment Policy

Assignments	Lab Mid Term Exam	Lab Terminal Exam	Total
25	25	50	100

Note: Midterm and Final term exams must be computer based.

List of Labs

LAB 01: Basics of OPENMP

Objective:

The objective of this lab is to teach the students about the ***Basics of OPENMP*** with the help of examples and learning tasks.

Activity Outcome:

In this lab, students will learn:

- Basics Of OpenMP
- Components of OpenMP API
- Setting up an OpenMP environment & its important terms
- Structure of OpenMp program

Instructor Note:

As pre-lab activity, read Basics of OPENMP Book Robert Chapter 7, Barlas: Chapter 4.

1) Useful Concepts

OpenMP

- OpenMP is a portable and standard Application Program Interface (API) that may be used to explicitly direct multi-threaded, shared memory parallelism
- OpenMP attempts to standardize existing practices from several different vendor-specific shared memory environments. OpenMP provides a portable shared memory API across different platforms including DEC, HP, IBM, Intel, Silicon Graphics/Cray, and Sun. The languages supported by OpenMP are FORTRAN, C and C++. Its main emphasis is on performance and scalability.

Goals of OpenMP

- **Standardization:** Provide a standard among a variety of shared memory architectures/platforms
- **Lean and Mean:** Establish a simple and limited set of directives for programming shared memory machines. Significant parallelism can be implemented by using just 3 or 4 directives.
- **Ease of Use:** Provide capability to incrementally parallelize a serial program, unlike message-passing libraries which typically require an all or nothing approach and provide the capability to implement both coarse-grain and fine-grain parallelism
- **Portability:** Supports Fortran (77, 90, and 95), C, and C++

OpenMP Programming Model:

Shared Memory, Thread Based Parallelism:

- OpenMP is based upon the existence of multiple threads in the shared memory programming paradigm. A shared memory process consists of multiple threads.

Explicit Parallelism:

- OpenMP is an explicit (not automatic) programming model, offering the programmer full control over parallelization.

Fork - Join Model:

- OpenMP uses the fork-join model of parallel execution:

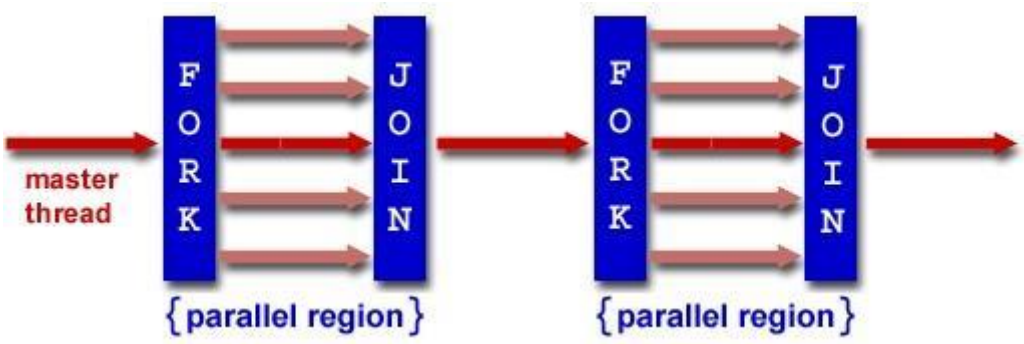


Figure 1 Fork and Join Model

- All OpenMP programs begin as a single process: the **master thread**. The master thread executes sequentially until the first **parallel region** construct is encountered.
- **FORK:** the master thread then creates a ***team*** of parallel threads
- The statements in the program that are enclosed by the parallel region construct are then executed in parallel among the various team threads
- **JOIN:** When the team threads complete the statements in the parallel region construct, they synchronize and terminate, leaving only the master thread

Compiler Directive Based:

- Most OpenMP parallelism is specified through the use of compiler directives which are imbedded in C/C++ or Fortran source code.

Nested Parallelism Support:

- The API provides for the placement of parallel constructs inside of other parallel constructs.
- Implementations may or may not support this feature.

Dynamic Threads:

- The API provides for dynamically altering the number of threads which may used to execute different parallel regions.
- Implementations may or may not support this feature.

I/O:

- OpenMP specifies nothing about parallel I/O. This is particularly important if multiple threads attempt to write/read from the same file.
- If every thread conducts I/O to a different file, the issues are not as significant.
- It is entirely up to the programmer to ensure that I/O is conducted correctly within the context of a multi-threaded program.

Components of OpenMP API

- Comprised of three primary API components:
- Compiler Directives
- Runtime Library
- Routines
- Environment Variables

2) Solved Lab Activities

Sr.No	*Allocated Time*	*Level of Complexity*	*CLO Mapping*
1	*30*	*Low*	*CLO-4*

Activity 1:

Write the General Code structure of A OpenMP program using C/C++.

C++Program Structure

Important terms for an OpenMP environment:

Construct: A construct is a directive or a combination of directives that defines a parallel region or specifies parallel execution of certain code blocks. These constructs provide a way for the programmer to indicate to the compiler which parts of the code should be parallelized and how parallelism should be managed.

Directive: A directive is a pragma or compiler directive that provides instructions to the compiler on how to parallelize and optimize the code. Directives are special annotations embedded within the source code of a program and are prefixed with **#pragma** followed by the **Omp #pragma omp** to indicate that they are OpenMP directives.

Region: a region refers to a block of code enclosed by specific directives that indicate parallel execution. These directives define sections of code where parallelism should be applied and how it should be managed by the compiler and runtime system.A dynamic extent.

```
#include <omp.h>
main ()
{
        int var1, var2, var3;
        Serial code
                        #pragma omp parallel private(var1, var2)
            shared(var3)
            {
                Parallel section executed by all threads
                    .
                    .
                    .
                All threads join master thread and disband
             }
         Resume serial code
  }
```

Dynamic extent: All statements in the *lexical extent*, plus any statement inside a function that is executed as a result of the execution of statements within the lexical extent. A dynamic extent is also referred to as a *region*.

Lexical extent: Statements lexically contained within a *structured block*.

Structured block: A structured block is a statement (single or compound) that has a single entry and a single exit. No statement is a structured block if there is a jump into or out of that statement. A compound statement is a structured block if its execution always begins at the opening **{** and always ends at the closing **}**. An expression statement, selection statement, iteration statement is a structured block if the corresponding compound statement obtained by enclosing it in **{** and **}**would be a structured block. A jump statement, labeled statement, or declaration statement is not a structured block.

Thread: An execution entity having a serial flow of control, a set of private variables, and access to shared variables.

Master thread: The thread that creates a team when a *parallel region* is entered.

Serial region: Statements executed only by the *master thread* outside of the dynamic extent of any *parallel region.*

Parallel region: Statements that bind to an OpenMP parallel construct and may be executed by multiple threads.

Variable: An identifier, optionally qualified by namespace names, that names an object.

Private: A private variable names a block of storage that is unique to the thread making the reference. Note that there are several ways to specify that a variable is private: a definition within a parallel region, a threadprivate directive, a private, firstprivate, lastprivate, or reduction clause, or use of the variable as a forloop control variable in a for loop immediately following a for or parallel for directive.

Shared: A shared variable names a single block of storage. All threads in a team that access this variable will access this single block of storage.

Team: One or more threads cooperating in the execution of a construct.

Serialize: To execute a parallel construct with a team of threads consisting of only a single thread (which is the master thread for that parallel construct), with serial order of execution for the statements within the structured block (the same order as if the block were not part of a parallel construct), and with no effect on the value returned by **omp_in_parallel()** (apart from the effects of any nested parallel constructs).

In OpenMP, the term "serialize" refers to a situation where a parallel construct is executed with only a single thread, essentially turning off parallel execution for that particular block of code. This can be useful in scenarios where parallel execution is not desired or appropriate, or when the overhead of parallelism outweighs the benefits.

Execute a parallel construct with a team of threads consisting of only a single thread: This means that even though the code is enclosed within a parallel construct (such as #pragma omp parallel), only one thread will be used to execute the parallel region.

Serial order of execution for the statements within the structured block: The statements within the parallel region will be executed in a serial manner, meaning they will be executed one after the other, just like they would be in a sequential code block.

No effect on the value returned by omp_in_parallel(): The function omp_in_parallel() is used to determine whether the calling thread is currently executing in a parallel region. In the case of serialization, although the code is within a parallel construct, since it's executed with only one thread, the function omp_in_parallel() will return false.

Barrier: A synchronization point that must be reached by all threads in a team. Each thread waits until all threads in the team arrive at this point. There are explicit barriers identified by directives and implicit barriers created by the implementation.

3) Lab Graded Tasks:

Lab Task 1

Discuss the different types of Parallel Programming Models.

Lab Task 2

List the possible characteristics of an APIs (Application programming interface).

Lab Task 3

Write the simple Hello World Program using OpenMp.

LAB 02: Socket Programming

Objective:

The objective of this lab is to teach the students about Socket **Programming** with the help of examples and learning tasks.

Activity Outcome:

In this lab, students will learn:

- Socket Programming basic concepts
- How it works
- "Passive" Listening Socket VS Active "Connected" Socket
- Socket Programming Using Python
- Simple Server and Client Connection

1) Useful Concepts

"Socket programming is a way of connecting two nodes on a network to communicate with each other." A *socket* is one endpoint of a two-way communication link between two programs running on the network. Socket acts as an interface between application and network. It supports basic network communications on an IP network. It is mostly used to **create a client-server environment**. Network Socket is used to identify processes (programs) on machines. It is composed of 2 numbers:

- IP address – machine identifier
- Port number – process identifier

There are two socket types for two transport services:

UDP:

SOCK_DGRAM is a datagram-based protocol that involves NO connection.
It is an unreliable service.

TCP:

SOCK_STREAM is a "reliable" or "confirmed" service. It is based on TCP, data transmission is more secure.

Socket State:

Sockets can be in different states depending on their lifecycle. Common states include listening, established, and closed.

How Does It Work?

The application creates a socket.
The socket type dictates the style of communication.
Once configured, the application can pass data to the socket for network transmission and receive data from the socket (transmitted through the network by some other host).

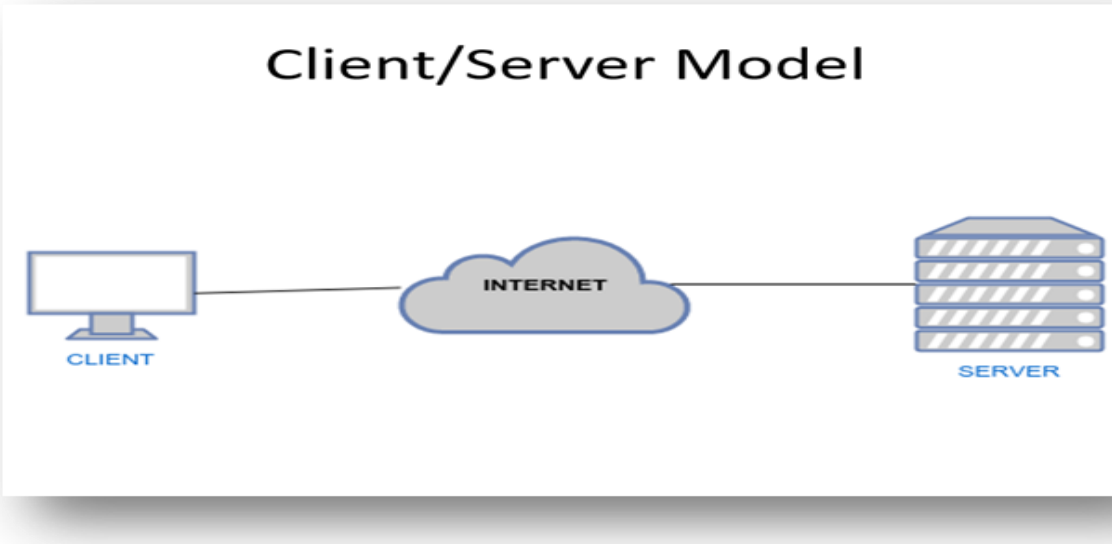


Figure 2. Client Server Model

“Passive” Listening Socket VS Active “Connected” Socket:

Server:
The server is a machine that is ready to receive connections. It holds the address and the port number.

Passive Listening Socket:
It DOES NOT represent any actual network conversation. No data can be received / sent by this kind of port! (Q: HOW it works then???)
When the server get the request from the passive listening socket, server checks with the OS willingness to receive incoming connections. If NO, drop the request. If YES, an active connected socket is then created. This socket is bounded to one remote conversation partner

Socket Programming Using Python

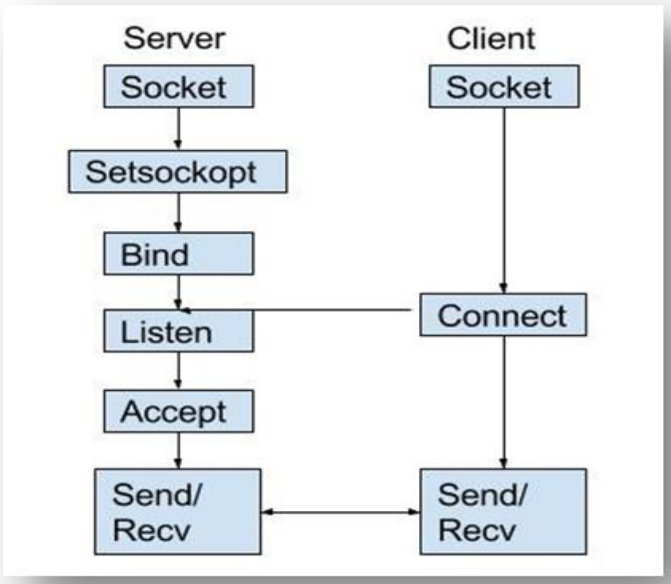


Figure 3. Client Server Connection Components

Socket(): To create a socket.

Setsockopt(): This function provides an application *program* with the means to control socket behavior.

Bind(): This method binds the address (hostname, port number) to the socket.

Listen(): Indicates a readiness to accept client connection requests

Accept(): Blocks and waits for an incoming connection.

Connect() : Connects client socket to a TCP based server socket.

2) Solved Lab Activities

Sr. No	*Allocated Time*	*Level of Complexity*	*CLO Mapping*
1	*20*	*Medium*	*CLO-4*
2	*20*	*Medium*	*CLO-4*

Activity 1: Create a simple client using socket programming concepts.

Client Code

```
import socket

# Define the host and port of the server to connect to

HOST = '127.0.0.1'  # The server's hostname or IP address

PORT = 65432            # The port used by the server

# Create a socket object

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:

 # Connect to the server

 s.connect((HOST, PORT))

 # Send data to the server

 s.sendall(b'Hello, server!')

 # Receive response from the server

data = s.recv(1024)

print('Received:', data.decode())
```

Activity 2: Create a simple Server using socket programming concepts to listen client request created in activity 1.

Server Side Code

```
import socket

# Define the host and port to listen on

 HOST = '127.0.0.1'  # Standard loopback interface address (localhost)

 PORT = 65432          # Port to listen on (non-privileged ports are > 1023)

# Create a socket object
```

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:

```
# Bind the socket to the address and port
s.bind((HOST, PORT))
```

 # Listen for incoming connections

```
        s.listen()
        print("Server is listening...")
```

 # Accept incoming connections

```
conn, addr = s.accept()
with conn:
print(f"Connected by {addr}")
# Receive data from the client
data = conn.recv(1024)
print(f"Received data: {data.decode()}")
```

 # Send a response back to the client

```
conn.sendall(b"Message received by the server")
 conn.close();
```

Simple server and client connection

AF_INET is an address family that is used to designate the type of addresses that your socket can communicate with.

SOCK_STREAM means that it is **a TCP socket (**SOCK_DGRAM for UDP**).**

Let's try another simple task for TCP server and client.

TCPserver.py TCPclient.py

```
import socket,sys
s= socket.socket(socket.AF_INET, socket.SOCK_STREAM)
HOST='127.0.0.1'
PORT=8888

def recv_all(sock, length):
    data=''
    while len(data)<length:
        more=sock.recv(length - len(data))
        if not more:
            print("......end")
            raise EOFError('socket closed %d bytes into a %d-byte message'
                           %(len(data), length))
        data+=more.decode("utf-8")
    return data

s.setsockopt(socket.SOL_SOCKET,socket.SO_REUSEADDR,1)
s.bind((HOST,PORT))
s.listen(1)
while True:
    print('Listening at',s.getsockname())
    sc, sockname=s.accept()
    print('We have accepted a connection from ',sockname)
    print('Socket connects',sc.getsockname(), 'and', sc.getpeername())
    message = recv_all(sc,16)
    print('The incoming 16-octect message says',repr(message))
    sc.sendall('BYE BYE CLIENT...'.encode("utf-8"))
    sc.close()
    print('Reply sent, socket closed')
```

Figure 4: TCP Server code in python

```
Listening at ('127.0.0.1', 8888)
We have accepted a connection from  ('127.0.0.1', 50454)
Socket connects ('127.0.0.1', 8888) and ('127.0.0.1', 50454)
The incoming 16-octect message says 'Hello!! Server!!'
Reply sent, socket closed
Listening at ('127.0.0.1', 8888)
```

Figure 5: TCP Server output

TCPserver.py TCPclient.py

```
import socket

s=socket.socket(socket.AF_INET,socket.SOCK_STREAM)

HOST= '127.0.0.1'
PORT = 8888

def recv_all(sock,length):
    data=''
    while len(data)<length:
        more= sock.recv(length-len(data))
        if not more:
            raise EOFError('socket closed %d bytes into a %d-byte'
                           'message' %(len(data),length))
        data+=more.decode("utf-8")
    return data

s.connect((HOST, PORT))
print('Client has been assigned socket name',s.getsockname())
msg='Hello!! Server!!'
s.sendall(msg.encode('utf-8'))
reply = recv_all(s,16)
print('THE SERVER SAID', repr(reply))
s.close()
```

```
Client has been assigned socket name ('127.0.0.1', 50454)
THE SERVER SAID 'BYE BYE CLIENT..'

Process finished with exit code 0
```

3) Lab Graded Tasks:

The above program comes with a limitation that the message must be fixed in 16 characters (octets)!

Your task is to modify the above program

- Determine the length of message L before sending it (from client side). [Assume max. number of length is 255 i.e. 3 digits], pad with leading zeros using zfill() method.
- Add L at the beginning of message.
- Send that message to server.
- Server should extract the length of message (first 3 characters) and read the rest of message with proper length.

Lab 03: Socket Programming with Multi-Threading

Objective:

The objective of this lab is to teach the students about Socket **Programming with Multithreading** with the help of examples and learning tasks.

Activity:

In this lab, students will learn:

- Socket Programming with Multi-Threading
- Multi-threading Modules

1) Useful Concepts

Socket Programming with Multi-threading

In this lab we will study about socket programming with multithreading. Let's revise a few basic concepts first.

Thread:

A thread is a light-weight process that does not require much memory overhead, they are cheaper than processes.

Multi-threading:

Multithreading is a process of executing multiple threads simultaneously in a single process.

Multi-threading Modules in Socket Programming:

Following are the modules that can be used to achieve multi-threading in python:

- _thread
- Threading

These modules help in synchronization and provides lock. Lock has two states "locked" and "unlocked".

acquire() is used to change state to locked

release() is used to change state to unlock.

Thread() is used to start a new thread in socket programming.
It takes 2 arguments, first is the function upon which we want to apply multi-threading and second is a tuple holding the address of client.

2) Solved Lab Activities

Sr.No	*Allocated Time*	*Level of Complexity*	*CLO Mapping*
1	*20*	*Medium*	*CLO-4*
2	*20*	*Medium*	*CLO-4*

Activity 1: Create a Multithreaded server.

Multi-threaded Server Code: At first step, import the necessary modules. Don't forget to call

```
print_lock = threading.Lock()
def createThread(c,addr ):
    print(f"new connection : {addr}")
    print(f"threadName : ", {threading.currentThread().getName()})
    while True:
        data = c.recv(1024)
        print('Received from the client :', str(data.decode('ascii')))
        if not data:
            print('Bye')
            print_lock.release()
            break
        data = "Today we will learn about Multithreading with Socket programming"
        c.send(data.encode('utf-8'))
    c.close()
def Main():
    host = "localhost"
    port = 2022
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.bind((host, port))
    print("socket binded to port", port)
    s.listen()
    print("Server is listening, Waiting for incoming ")
    while True:
        c, addr = s.accept()
        print_lock.acquire()
        print('Connected to :', addr[0], ':', addr[1])
        print(f"threadName , before creating thread: : ", {threading.currentThread().getName()})
        thread= threading.Thread(target=createThread, args=(c,addr))
        thread.start()
    s.close()
```

Main().

Activity 2: Create a Multithreaded Client.

Client Code:

```
import socket

host = '127.0.0.1'

port = 2022
s = socket.socket(socket.AF_INET,socket.SOCK_STREAM)
s.connect((host,port))
message = "Hey Server!!! What will we do in today's lab???"
while True:
    s.send(message.encode('utf-8'))
    data = s.recv(1024)
    print('Received from the server :',str(data.decode('utf-8')))
    reply = input('\nDo you want to continue(y/n) :')
    if reply == 'y':
        continue
    else:
        print("Bye Server")
        break
s.close()
```

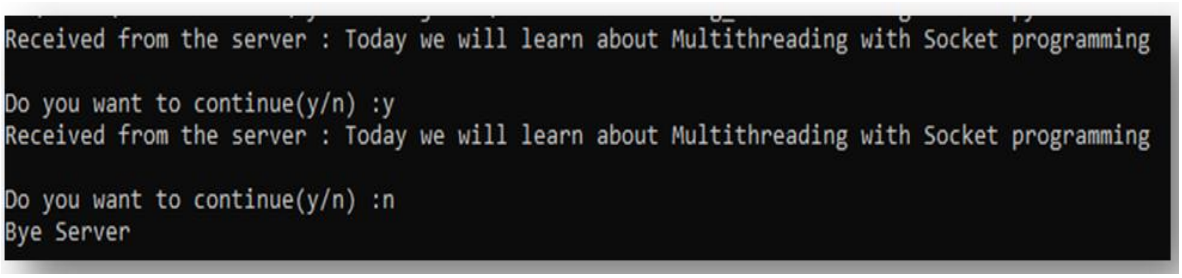
Output of Server code:

```
socket binded to port 2022
Server is listening, Waiting for incoming
Connected to : 127.0.0.1 : 50376
threadName , before creating thread: :  {'MainThread'}
new connection : ('127.0.0.1', 50376)
threadName :  {'Thread-1'}
Received from the client : Hey Server!!! What will we do in today's lab???
Received from the client :
Bye
Connected to : 127.0.0.1 : 50377
threadName , before creating thread: :  {'MainThread'}
new connection : ('127.0.0.1', 50377)
threadName :  {'Thread-2'}
Received from the client : Hey Server!!! What will we do in today's lab???
Received from the client :
Bye
```

Client with port number 50376 is served by “Thread-1”

Client with port number 50377 is served by “Thread-2”

Output of Client/s Code:


Received from the server : Today we will learn about Multithreading with Socket programming
Do you want to continue(y/n) :y
Received from the server : Today we will learn about Multithreading with Socket programming
Do you want to continue(y/n) :n
Bye Server

Lab 04 (a): Sharing of work among threads using Loop Construct in OpenMP

Objective:

The objective of this lab is to teach the students about the Sharing of work among threads using Loop Construct in OpenMP with the help of examples and learning tasks.

Activity Outcomes:

In this lab, students will learn how to:

- The Loop Construct of OpenMP
- Combined Parallel Work-Sharing Constructs
- Steps for Parallel Programming

1) Useful Concepts

Introduction

OpenMP's work-sharing constructs are the most important feature of OpenMP. They are used to distribute computation among the threads in a team. C/C++ has three work-sharing constructs. A work-sharing construct, along with its terminating construct where appropriate, specifies a region of code whose work is to be distributed among the executing threads; it also specifies the manner in which the work in the region is to be parceled out. A work-sharing region must bind to an active parallel region in order to have an effect. If a work-sharing directive is encountered in an inactive parallel region or in the sequential part of the program, it is simply ignored. Since work- may occur in procedures that are invoked both from within a parallel region as well as outside of any parallel regions, they may be exploited during some calls and ignored during others.
sharing directives
The work-sharing constructs are listed below.

#pragma omp for: directive is used in OpenMP to parallelize a loop in C or C++ code. It instructs the compiler to distribute the iterations of the loop among multiple threads, allowing for parallel execution.

#pragma omp sections: Distribute independent work units

- **#pragma omp single:** Only one thread executes the code block

The two main rules regarding work-sharing constructs are as follows:
Each work-sharing region must be encountered by all threads in a team or by none at all.

- The sequence of work-sharing regions and barrier regions encountered must be the same for every thread in a team.

A work-sharing construct does not launch new threads and does not have a barrier on entry. By default, threads wait at a barrier at the end of a work-sharing region until the last thread has completed its share of the work. However, the programmer can suppress this by using the nowait clause.

The Loop Construct

The *loop construct* causes the iterations of the loop immediately following it to be executed in parallel. At run time, the loop iterations are distributed across the threads. This is probably the most widely used of the work-sharing features.

Format:

#pragma omp for [clause ...] newline //This line is the start of the OpenMP directive. It indicates that the following loop should be parallelized.

schedule (type [,chunk]) ordered
private (list) firstprivate (list) lastprivate (list)
shared (list) reduction (operator: list) nowait
for_loop

2) Solved Lab Activities

Sr. No	*Allocated Time*	*Level of Complexity*	*CLO Mapping*
1	*25*	*Low*	*CLO-4*
2	*30*	*medium*	*CLO-4*

Activity 1: Create a program using work-sharing loop.
Example of a work-sharing loop

Each thread executes a subset of the total iteration space $i = 0, \ldots, n - 1$

```
#include      <omp.h>
main()
{
        #pragma omp parallel shared(n) private(i)
        {
                #pragma omp for for (i=0;
                i<n;i++)
                printf("Thread %d executes loop iteration %d\n",
                omp_get_thread_num(),i);
        }
}
```

Here we use a parallel directive to define a parallel region and then share its work among threads via the for-work-sharing directive: the **#pragma omp for** directive states that iterations of the loop following it will be distributed. Within the loop, we use the OpenMP function omp_get_thread_num(), this time to obtain and print the number of the executing thread in each iteration. Parallel construct that state which data in the region is shared and which is private. Although not strictly needed since this is enforced by the compiler, loop variable i is explicitly declared to be a private variable, which means that each thread will have its own copy of i. its value is also undefined after the loop has finished. Variable n is made shared.
Output from the example which is executed for $n = 9$ and uses four threads.

```
Thread 0 executes loop iteration 0
Thread 0 executes loop iteration 1
Thread 0 executes loop iteration 2
Thread 3 executes loop iteration 7
Thread 3 executes loop iteration 8
Thread 2 executes loop iteration 5
Thread 2 executes loop iteration 6
Thread 1 executes loop iteration 3
Thread 1 executes loop iteration 4
```

Activity 2: Create a matrix multiplication program using Combined Parallel Work-Sharing Constructs.

Activity Combined Parallel Work-Sharing Constructs

Combined parallel work-sharing constructs are shortcuts that can be used when a parallel region comprises precisely one work-sharing construct, that is, the work-sharing region includes all the code in the parallel region. The semantics of the shortcut directives are identical to explicitly specifying the parallel construct immediately followed by the work-sharing construct.

Full version Combined construct	Combined construct

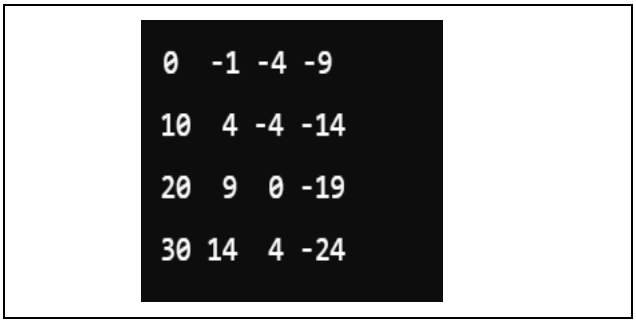
#pragma omp parallel
{
#pragma omp for for-loop
}
	#pragma omp parallel for
	{
	for-loop
	}

Solution Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

#define N 4  // Define the size of the matrix
void print_matrix(int matrix[N][N]) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            printf("%d ", matrix[i][j]);
        }
        printf("\n");
    }
}

int main() {
    int A[N][N], B[N][N], C[N][N];

    // Initialize matrices A and B with some values
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            A[i][j] = i + j;  // Example initialization
            B[i][j] = i - j;  // Example initialization
        }
    }

    // Print the input matrices
    printf("Matrix A:\n");
    print_matrix(A);
    printf("\nMatrix B:\n");
    print_matrix(B);

    // Matrix multiplication using OpenMP
    #pragma omp parallel
    {
        #pragma omp for
        for (int i = 0; i < N; i++) {
            for (int j = 0; j < N; j++) {
                C[i][j] = 0;
                for (int k = 0; k < N; k++) {
                    C[i][j] += A[i][k] * B[k][j];
                }
            }
        }
```

```
    }

    // Print the resulting matrix C
    printf("\nMatrix C (Result of A * B):\n");
    print_matrix(C);

    return 0;
}
```

Output



3) Lab Graded Tasks:

1) *Execute the above program code and write its output.*
2) *Write a simple parallel program, which uses Loop Construct.*

Lab 04 (b): Clauses in Loop Construct

1) Useful Concepts

Introduction

The OpenMP Data Scope Attribute Clauses are used to explicitly define how variables should be scoped. Data Scope Attribute Clauses are used in conjunction with several directives (PARALLEL, DO/for, and SECTIONS) to control the scoping of enclosed variables. These constructs provide the ability to control the data environment during execution of parallel constructs. They define how and which data variables in the serial section of the program are transferred to the parallel sections of the program (and back). They define which variables will be visible to all threads in the parallel sections and which variables will be privately allocated to all threads.

List of Clauses

PRIVATE

FIRSTPRIVATE

LASTPRIVATE

SHARED

DEFAULT

REDUCTION

COPYIN

PRIVATE Clause

The PRIVATE clause declares variables in its list to be private to each thread.

Format: PRIVATE (list)

Notes:

PRIVATE variables behave as follows:
A new object of the same type is declared once for each thread in the team
All references to the original object are replaced with references to the new object
Variables declared PRIVATE are uninitialized for each thread

2) Solved Lab Activities

Sr.No	*Allocated Time*	*Level of Complexity*	*CLO Mapping*
1	***10***	***Low***	***CLO-4***
2	***15***	***Low***	***CLO-4***
3	***15***	***Medium***	***CLO-4***
4	***20***	***Medium***	***CLO-4***
5	***20***	***Medium***	***CLO-4***
6	***15***	***Medium***	***CLO-4***

Activity 1:

Example of the private clause –

```
Each thread has a local copy of variables i and a.

        #pragma omp parallel for private(i,a)

                for (i=0; i<n; i++)
                { a = i+1;
printf("Thread %d has a value of a = %d for i = %d\n", omp_get_thread_num(),a,i); } /*-- End of
parallel for --*/
```

It is the programmer's responsibility to ensure that multiple threads properly access SHARED variables (such as via CRITICAL sections)

SHARED Clause

The SHARED clause declares variables in its list to be shared among all threads in the team. **Format:** SHARED *(list)*

Notes: A shared variable exists in only one memory location and all threads can read or write to that address.

Activity 2:

Example of the shared clause – All threads can read from and write to vector a.

```
        #pragma omp parallel for shared(a)

                for (i=0; i<n; i++)
                { a[i] += i;
                } /*-- End of parallel for --*/
```

DEFAULT Clause

The DEFAULT clause allows the user to specify a default PRIVATE, SHARED, or NONE scope for all variables in the lexical extent of any parallel region.

The default clause is used to give variables a default data-sharing attribute. Its usage is straightforward. For example, default (shared) assigns the shared attribute to all variables referenced in the construct. This clause is most often used to define the data-sharing attribute of the majority of the variables in a parallel region. Only the exceptions need to be explicitly listed.
If default (none) is specified instead, the programmer is forced to specify a data-sharing attribute for each variable in the construct. Although variables with a predetermined datasharing attribute need not be listed in one of the clauses, it is strongly recommend that the attribute be explicitly specified for *all* variables in the construct.
Format: DEFAULT (SHARED | NONE)

Notes:
Specific variables can be exempted from the default using the PRIVATE, SHARED, FIRSTPRIVATE, LASTPRIVATE, and REDUCTION clauses
The C/C++ OpenMP specification does not include "private" as a possible default.
However, actual implementations may provide this option.
Only one DEFAULT clause can be specified on a PARALLEL directive

Example of the Default clause: all variables to be shared, with the exception of a, b, and c.
#pragma omp for default(shared) private(a,b,c)

FIRSTPRIVATE Clause

The FIRSTPRIVATE clause combines the behavior of the PRIVATE clause with automatic initialization of the variables in its list.
Format: FIRSTPRIVATE *(LIST)*

Notes:
Listed variables are initialized according to the value of their original objects prior to entry into the parallel or work-sharing construct

Activity 3:

Example using the firstprivate clause – Each thread has a pre-initialized copy of variable indx. This variable is still private, so threads can update it individually.

```
for(i=0; i<vlen; i++) a[i] = -i-1; indx = 4;
{
#pragma omp parallel default(none) firstprivate(indx) private(i,TID) shared(n,a)
 {
                TID = omp_get_thread_num(); indx += n*TID;
                for(i=indx; i<indx+n; i++) a[i] = TID + 1;
        }
}
printf("After the parallel region:\n"); for (i=0; i<vlen; ;i++)

printf("a[%d] = %d\n",i,a[i]);
```

LASTPRIVATE Clause

The LASTPRIVATE clause combines the behavior of the PRIVATE clause with a copy from the last loop iteration or section to the original variable object.

Format: LASTPRIVATE *(LIST)*

Notes:

- The value copied back into the original variable object is obtained from the last (sequentially) iteration or section of the enclosing construct.
- It ensures that the last value of a data object listed is accessible after the corresponding construct has completed execution

Activity 4:

Example of the lastprivate clause – This clause makes the sequentially last value of variable a accessible outside the parallel loop.

```
#pragma omp parallel for private(i) lastprivate(a) for (i=0; i<n; i++)
        {
         a = i+1;
                printf("Thread %d has a value of a = %d for i =
                %d\n",
        omp_get_thread_num(),a,i);
        } /*-- End of parallel for --*/

        printf("Value of a after parallel for: a = %d\n",a);
```

COPYIN Clause

The COPYIN clause provides a means for assigning the same value to THREADPRIVATE variables for all threads in the team.

Format: COPYIN *(LIST)*

Notes:
List contains the names of variables to copy. The master thread variable is used as the copy source. The team threads are initialized with its value upon entry into the parallel construct.

REDUCTION Clause
The REDUCTION clause performs a reduction on the variables that appear in its list. A private copy for each list variable is created for each thread. At the end of the reduction, the reduction variable is applied to all private copies of the shared variable, and the final result is written to the global shared variable.

Format: REDUCTION *(OPERATOR: LIST)*
Notes:

- Variables in the list must be named scalar variables. They cannot be array or structure type variables. They must also be declared SHARED in the enclosing context.
- Reduction operations may not be associative for real numbers.
- The REDUCTION clause is intended to be used on a region or work-sharing construct in which the reduction variable is used only in statements which have one of following forms:

C / C++

`x = x op expr`
`x = expr op x` (except subtraction)
`x binop = expr`
`x++ ++x`
`x-- --x`

x is a scalar variable in the list ***expr*** is a scalar expression that does not reference ***x*** ***op*** is not overloaded, and is one of +, *, -, /, &, ^, |, &&, || ***binop*** is not overloaded, and is one of +, *, -, /, &, ^, |

Activity 5:

Example of REDUCTION - Vector Dot Product. Iterations of the parallel loop will be distributed in equal sized blocks to each thread in the team (SCHEDULE STATIC). At the end of the parallel loop construct, all threads will add their values of "result" to update the master thread's global copy.

```
#include <omp.h> main ()
{ int   i, n, chunk; float a[100], b[100], result;
        n = 100; chunk = 10; result = 0.0;

        for (i=0; i < n; i++)
          {
          a[i] = i * 1.0;   b[i] = i *
        2.0;
          }
        #pragma omp parallel for default(shared) private(i)  schedule(static,chunk)
        reduction(+:result)
          {
            for (i=0; i < n; i++)
            result = result + (a[i] * b[i]);

printf("Final result= %f\n",result); }
}
```

SCHEDULE:

Describes how iterations of the loop are divided among the threads in the team. The default schedule is implementation dependent.

STATIC

Loop iterations are divided into pieces of size chunk and then statically assigned to threads. If chunk is not specified, the iterations are evenly (if possible) divided contiguously among the threads.

DYNAMIC

Loop iterations are divided into pieces of size chunk, and dynamically scheduled among the threads; when a thread finishes one chunk, it is dynamically assigned another. The default chunk size is 1.

GUIDED

For a chunk size of 1, the size of each chunk is proportional to the number of unassigned iterations divided by the number of threads, decreasing to 1. For a chunk size with value k (greater than 1), the size of each chunk is determined in the same way with the restriction that the chunks do not contain fewer than k iterations (except for the last chunk to be assigned, which may have fewer than k iterations). The default chunk size is 1.

Nowait Clause

The nowait clause allows the programmer to fine-tune a program's performance. When we introduced the work-sharing constructs, we mentioned that there is an implicit barrier at the end of them. This clause overrides that feature of OpenMP; in other words, if it is added to a construct, the barrier at the end of the associated construct will be suppressed. When threads reach the end of the construct, they will immediately proceed to perform other work. Note, however, that the barrier at the end of a parallel region cannot be suppressed.

Activity 6:

Example of the nowait clause in C/C++ – The clause ensures that there is no barrier at the end of the loop.

```
 #include <stdio.h>
#include <omp.h>

#define N 10  // Size of the array

int main() {
   int array[N];

   // Initialize the array
   for (int i = 0; i < N; i++) {
      array[i] = 0;
   }

   #pragma omp parallel
   {
      // First parallel region: perform a
         task with a loop
      #pragma omp for
      for (int i = 0; i < N; i++) {
         array[i] = i * i;
         printf("First loop iteration:
         array[%d] = %d\n", i,
         array[i]);
      }

      // Do additional work in parallel
         without waiting for the
         previous loop to complete
      #pragma omp for nowait
      for (int i = 0; i < N; i++) {
         array[i] += 5;
         printf("Second loop iteration:
         array[%d] = %d\n", i,
         array[i]);
      }
   }

   return 0;
}
```

Clauses / Directives Summary

Table 1 Comparative Analysis for a set of instruction

Clause	Directive					
	PARALLEL	DO/for	SECTIONS	SINGLE	PARALLEL DO/for	PARALLEL SECTIONS
IF	●				●	●
PRIVATE	●	●	●	●	●	●
SHARED	●	●			●	●
DEFAULT	●				●	●
FIRSTPRIVATE	●	●	●	●	●	●
LASTPRIVATE		●			●	●
REDUCTION	●	●	●		●	●
COPYIN	●				●	●
SCHEDULE		●			●	
ORDERED		●			●	
NOWAIT		●	●	●		

The following OpenMP directives do not accept clauses:

MASTER

CRITICAL

BARRIER

ATOMIC

FLUSH

ORDERED

THREADPRIVATE

Implementations may (and do) differ from the standard in which clauses are supported by each directive.

3) Lab Graded Tasks:

1. Code the above example programs and write down their outputs.

2. Write a parallel program that sums a given arrays using reduction clause.

Lab 04 (c): Parallelizing a Loop with Scheduling

1) Useful Concepts

We use the **schedule** clause to specify how loop iterations are distributed among threads. The **dynamic** scheduling allows the scheduler to assign a new iteration to the next available thread.

Objective: Understand and experiment with loop scheduling in OpenMP to parallelize a loop with different scheduling options.

2) Solved Lab Activities

Sr. No	*Allocated Time*	*Level of Complexity*	*CLO Mapping*
1	*20*	*Low*	*CLO-4*

```
#include <stdio.h>
#include <omp.h>

int main() {
    int n = 100;
    int result[n];

    #pragma omp parallel
    {
        // Experiment with different scheduling options: static, dynamic, guided
        #pragma omp for schedule(dynamic)
        for (int i = 0; i < n; i++) {
            int thread_id = omp_get_thread_num();
            result[i] = thread_id * n + i;
        }
    }

    printf("Results with dynamic scheduling:\n");
    for (int i = 0; i < n; i++) {
        printf("%d ", result[i]);
    }
    printf("\n");

    return 0;
}
```

Explanation:

The code sets up a parallel region using **#pragma omp parallel**.
Inside the parallel region, we parallelize a loop with dynamic scheduling using **#pragma omp for schedule(dynamic)**.
The loop assigns unique values to the elements of the **result** array based on thread ID and loop iteration.

3) Lab Graded Tasks:

1) *Compile and run the program to observe the output with dynamic scheduling.*
2) *Experiment with different scheduling options (static, dynamic, guided) by changing the schedule clause in the code.*
3) *Observe how changing the scheduling strategy affects the order in which loop iterations are executed.*

4) *Discuss the differences between static, dynamic, and guided scheduling.*
5) *Analyze how different scheduling strategies impact the order of execution and potentially improve load balancing in parallel loops.*

Lab 05: Threads Work-sharing for OpenMP program using ‘Sections Construct

Objective:

The objective of this lab is to teach the students about the Threads Work-sharing for OpenMP program using ‘Sections Construct’ with the help of examples and learning tasks.

Activity Outcomes:

In this lab, students will learn:
Introduction to OpenMP Sections Construct
Combined Parallel Work-Sharing Constructs

1) Useful Concepts

Introduction

OpenMP's work-sharing constructs are the most important feature of OpenMP. They are used to distribute computation among the threads in a team. C/C++ has three work-sharing constructs. A work-sharing construct, along with its terminating construct where appropriate, specifies a region of code whose work is to be distributed among the executing threads; it also specifies the manner in which the work in the region is to be parceled out. A work-sharing region must bind to an active parallel region in order to have an effect. If a work-sharing directive is encountered in an inactive parallel region or in the sequential part of the program, it is simply ignored. Since work-sharing directives may occur in procedures that are invoked both from within a parallel region as well as outside of any parallel regions, they may be exploited during some calls and ignored during others.

The work-sharing constructs are listed below.

- **#pragma omp for !$omp do** Distribute iterations over the threads
- **#pragma omp sections !$omp sections** Distribute independent work units
- **#pragma omp single !$omp single** Only one thread executes the code block

The two main rules regarding work-sharing constructs are as follows:
Each work-sharing region must be encountered by all threads in a team or by none at all.
The sequence of work-sharing regions and barrier regions encountered must be the same for every thread in a team.
A work-sharing construct does not launch new threads and does not have a barrier on entry. By default, threads wait at a barrier at the end of a work-sharing region until the last thread has completed its share of the work. However, the programmer can suppress this by using the nowait clause.

The Sections Construct

The *sections construct* is the easiest way to get different threads to carry out different kinds of work, since it permits us to specify several different code regions, each of which will be executed by one of the threads. It consists of two directives: first, **#pragma omp sections:** to indicate the start of the construct and second, **the #pragma omp section:** to mark each distinct section. Each section must be a structured block of code that is independent of the other sections.

At run time, the specified code blocks are executed by the threads in the team. Each thread executes one code block at a time, and each code block will be executed exactly once. If there are fewer threads than code blocks, some or all of the threads execute multiple code blocks. If there are fewer code blocks than threads, the remaining threads will be idle. Note that the assignment of code blocks to threads is implementation-dependent.

Format:

```
#pragma omp sections [clause ...] newline                private (list)
firstprivate (list)          lastprivate (list)          reduction (operator:
list)          nowait
        {
        #pragma omp section  newline          structured_block
        #pragma omp section  newline     structured_block
    }
```

Combined Parallel Work-Sharing Constructs

Combined parallel work-sharing constructs are shortcuts that can be used when a parallel region comprises precisely one work-sharing construct, that is, the work-sharing region includes all the code in the parallel region. The semantics of the shortcut directives are identical to explicitly specifying the parallel construct immediately followed by the worksharing construct.

Full version Combined construct	Combined construct

#pragma omp parallel
{
#pragma omp sections
{
[**#pragma omp section**] *structured block*
[**#pragma omp section** *structured block*
}
}
	#pragma omp parallel sections
	{
	[**#pragma omp section**] *structured block*
	[**#pragma omp section]** *structured block*
	. . .
	}

2) Solved Lab Activities

Sr.No	*Allocated Time*	*Level of Complexity*	*CLO Mapping*
1	*25*	*Low*	*CLO-4*

Activity 1: Write a program to that include parallel sections

Example program of parallel sections

If two or more threads are available, one thread invokes funcA() and another thread calls funcB(). Any other threads are idle.

```
#include <omp.h>
        main()
        {
                #pragma omp parallel
                {
                        #pragma omp sections
                        {
                                #pragma omp section
                                (void) funcA();

                                #pragma omp section
                                (void) funcB();

                         } /*-- End of sections block --*/

                  } /*-- End of parallel region --*/

        } /*-- End of Main Program --*/
void funcA()
                {
                printf("In funcA: this section is executed by thread
                %d\n",
                omp_get_thread_num());
                }
         void funcB()
                {
                printf("In funcB: this section is executed by thread
                %d\n",
                omp_get_thread_num());
        }
```

Output from the example; the code is executed by using two threads.

```
In funcA: this section is executed by thread 0
In funcB: this section is executed by thread
```

3) Lab Graded Tasks:

1. Code the above example programs and write down their outputs.

2. write a parallel program, after discussing your instructor, which uses Sections Construct.

LAB 06: Parallel Processing

Objective:

The objective of this lab is to teach the students about the Parallel Processing with the help of examples and learning tasks.

Activity Outcomes:

In this lab, students will learn:

- Parallel Processing concepts
- Handling parallel programs:

1) Useful Concepts

Introduction:

Parallel Processing is the simultaneous execution of a task through multiprocessors attached to the same computer. It involves **taking a process, dividing it into several smaller processes, and then working on each of those simultaneously**. The goal of this divide-and-conquer approach is to complete the larger task in less time than it would have taken to do it in one large chunk. In python, the multiprocessing module is used to run independent parallel processes.
By the end of this lab, you'll be able to:

Structure the code and enable parallel processing to parallelize any typical logic using python's multiprocessing.
Implement synchronous and asynchronous parallel processing.
Before moving on to the lab tasks, let's revise some basic concepts.

There are two main ways to handle parallel programs:

Shared Memory:

All the processors can access all memory locations and can read or write shared variables. The advantage is that you don't need to handle the communication explicitly. But here a problem arises when multiple processors access and change the same memory location at the same time. To avoid this conflict, synchronization techniques are used.

Distributed memory

In distributed memory, each process is totally separated and has its own memory space. In this scenario, communication is handled explicitly between the processes, so it is costlier compared to shared memory.

Multiprocessing for parallel processing

Using the standard multiprocessing module, we can efficiently parallelize simple tasks.
Maximum parallel processes you can run is limited by the number of processors in your computer.

```
import multiprocessing as mp
print("Number of processors: ", mp.cpu_count())
```

Synchronous Execution: A synchronous execution is one in which the processes are completed in the same order in which it was started. It means that the first task in a program must finish processing before moving on to executing the next task. This is achieved by locking the main program until the respective processes are finished.
Asynchronous Execution: When you run something asynchronously it means it is non-blocking, you execute it without waiting for it to complete and carry on with other things. As a result, the order of results can get mixed up but usually gets done quicker.

Python multiprocessing module provides pool class and process class.

Process class:

There are two important functions that belongs to the Process class – start() and join() function.

Start():
To start a process, we use start method of Process class.

Join():
The join method blocks the execution of the main process until the process whose join method is called terminates.

First we need to instantiate a process object. Process can be instantiated with or without arguments. If we want to pass any argument through the process use args keyword.

```
Process(target=function_name)# without arguments
Process(target=function_name, args=(name,)) #with arguments
```

When a process object will be created, nothing will happen until we tell it to start processing via start() function. The process will then run and return its results.
After that we tell the process to complete via join() function.
(So if you create many processes and don't terminate them, you may face scarcity of resources.)

2) Solved Lab Activities

Sr.No	*Allocated Time*	*Level of Complexity*	*CLO Mapping*
1	*25*	*Low*	*CLO-4*

Let's implement the following code to understand the Process class.

Pool class

```
import multiprocessing
from multiprocessing import Process

def print_func(language='Python'):
    curr_proc = multiprocessing.current_process()
    print('Language is : ', language, " and ", curr_proc.name, " is executing this function")

if __name__ == "__main__":
    names = ['Python', 'Java', 'JavaScript', 'Kotlin', 'C++']
    procs = []
    # instantiating without any argument
    proc = Process(target=print_func)
    procs.append(proc)
    proc.start()
    # instantiating process with arguments
    for name in names:
        proc = Process(target=print_func, args=(name,))
        procs.append(proc)
        proc.start()
    # complete the processes
    for proc in procs:
        proc.join()
```

The multiprocessing. Pool() class produces a set of processes called workers and can submit tasks using methods given in the table below.

Synchronous Execution	Asynchronous Execution
apply()	apply_async()
map()	map_async()
starmap()	starmap_async()

For parallel mapping, you should first initialize a multiprocessing.Pool() object. The first argument is the number of workers; if not given, that number will be equal to the number of cores in the system

```
from multiprocessing import Pool
import time

def f(x):
    return x*x

if __name__ == '__main__':
    pool = Pool(processes=4)
    for x in range(30000):
        pool.apply(f, (x,))
```

apply_async ():

```python
def f(x):

    curr_proc = multiprocessing.current_process()
    print(" ",x*x," is executing by : ",curr_proc.name)

if __name__ == '__main__':
    curr_proc = multiprocessing.current_process()
    pool = Pool(processes=4)
    for x in range(30000):
        r = pool.map_async(f, (x,))
    print ('\nHERE : process is :',curr_proc.name,"\n")
    print ('\nMORE : process is :',curr_proc.name,"\n")
    r.wait()
    print ('\nDONE process is :',curr_proc.name,"\n")
```

4) Lab Graded Tasks:

1) *Check the same code for the remaining methods i.e. map/map_async and starmap/starmap_async.*
2) *Also note down the execution time (for all methods)*
3) *Parallelize data processing with multiprocessing Queue*
4) *Calculate the square of numbers from 1 to 10 million using multiprocessing to utilize multiple CPU cores efficiently.*

LAB 07: Basics of MPI

Objective:

The objective of this lab is to teach the students about the Basics of MPI with the help of examples and learning tasks.

Outcomes:

In this lab, students will learn:

- Basics of MPI
- Structure of MPI Program

1) Useful Concepts

A Distributed system consists of a collection of autonomous computers, connected through a network and distribution middleware, which enables computers to coordinate their activities and to share the resources of the system so that users perceive the system as a single, integrated computing facility.
Let us say about Google Web Server, from users perspective while they submit the searched query, they assume google web server as a single system. However, behind the curtain, Google has built a lot of servers which is distributed (geographically and computationally) to give us the result within a few seconds.

Message Passing Interface (MPI) is a standardized and portable **message-passing** system developed for distributed and parallel computing. MPI provides parallel hardware vendors with a clearly defined base set of routines that can be efficiently implemented.
MPI gives users the flexibility of calling a set of routines from **C, C++, Fortran, C#, Java, or Python.**
The advantages of MPI over older message passing libraries are portability (because MPI has been implemented for almost every distributed memory architecture) and speed (because each implementation is in principle optimized for the hardware on which it runs)

Intallation for Python

sudo apt-get install libcr-dev mpich mpich-doc
sudo pip install mpi4py
sudo apt-get install python-dev

It is important to observe that when a program running with MPI, all processes use the same compiled binary, and hence all processes are running the exact same code. What in an MPI distinguishes a parallel program running on P processors from the serial version of the code running on P processors? Two things distinguish the parallel program

- Each process uses its process rank to determine what part of the algorithm instructions are meant for it.
- Processes communicate with each other in order to accomplish the final task.

Even though each process receives an identical copy of the instructions to be executed, this does not imply that all processes will execute the same instructions. Because each process is able to obtain its process rank (using MPI_Comm_rank).It can determine which part of the code it is supposed to run. This is accomplished through the use of IF statements. Code that is meant to be run by one particular process should be enclosed within an IF statement, which verifies the process identification number of the process. If the code is not placed with in IF statements specific to a particular id, then the code will be executed by all processes. The second point, communicating between processes; MPI communication can be summed up in the concept of sending and receiving messages. Sending and receiving is done with the following two functions: MPI Send and MPI Recv.

MPI_Send

```
int MPI_Send( void* message /* in */, int count /* in */, MPI Datatype datatype /* in */, int dest /* in
*/, int tag /* in
*/, MPI Comm comm /* in */ )
```

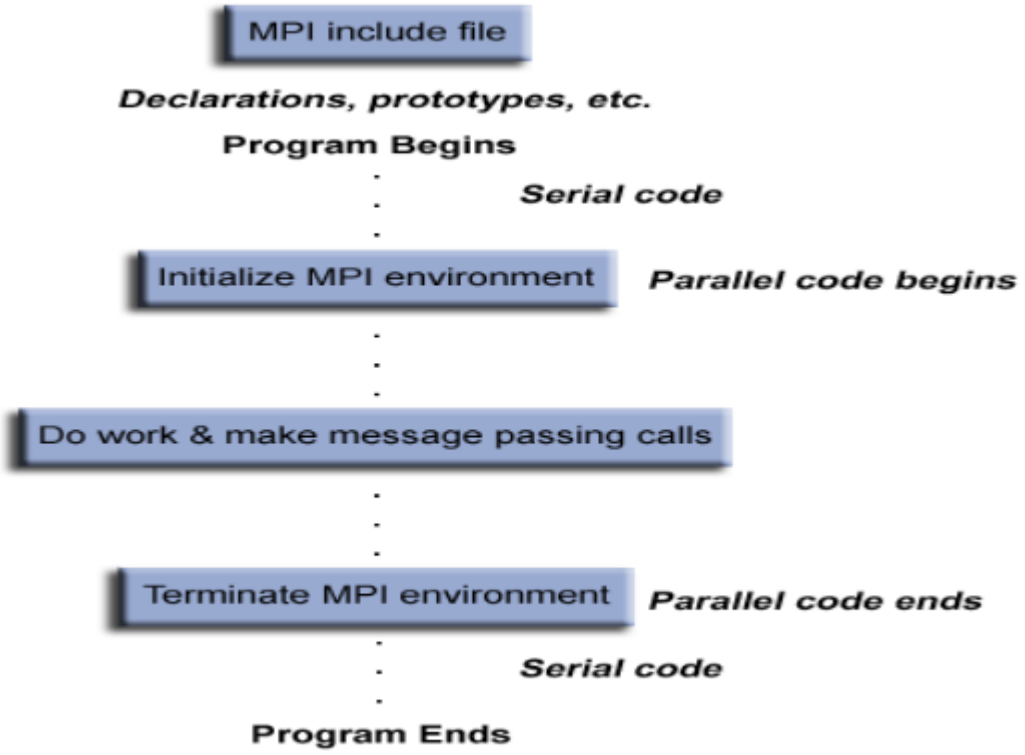
MPI_Recv

```
int MPI_Recv( void* message /* out */, int count /* in */, MPI
Datatype datatype /* in */, int source /* in */, int tag /* in
*/, MPI Comm comm /* in */, MPI Status* status /* out */)
```

Understanding the Argument Lists

message - starting address of the send/recv buffer.
count - number of elements in the send/recv buffer.
datatype - data type of the elements in the send buffer.
source - process rank to send the data.
dest - process rank to receive the data.
tag - message tag.
comm - communicator.
status - status object.

Structure of MPI Program



2) Solved Lab Activities

Sr. No	*Allocated Time*	*Level of Complexity*	*CLO Mapping*
1	*20*	*Low*	*CLO-4*
2	*20*	*Medium*	*CLO-4*

Activity 1:

An Example Program:

The following program demonstrate the use of send/receive function in which sender is initialized as node two (2) where as receiver is assigned as node four (4). The following program requires that it should be accommodated on five (5) nodes otherwise the sender and receiver should be initialized to suitable ranks.

```
#include <iostream.h>
#include <mpi.h>

int main(int argc, char ** argv
{ int mynode, totalnodes;
int datasize; // number of data units to be sent/recv int sender=2; // process number of the sending process int
receiver=4; // process number of the receiving process int tag; // integer message tag
MPI_Status status; // variable to contain status information

MPI_Init(&argc,&argv);

MPI_Comm_size(MPI_COMM_WORLD, &totalnodes);
MPI_Comm_rank(MPI_COMM_WORLD, &mynode);

// Determine datasize
double * databuffer = new double[datasize];
// Fill in sender, receiver, tag on sender/receiver processes,
// and fill in databuffer on the sender process.

if(mynode==sender)

MPI_Send(databuffer,datasize,MPI_DOUBLE,receiver,
tag,MPI_COMM_WORLD);

if(mynode==receiver)
    MPI_Recv(databuffer,datasize,MPI_DOUBLE,sender,tag,
MPI_COMM_WORLD,&status);
// Send/Recv complete              MPI_Finalize();
}
```

Key Points:

In general, the *message* array for both the sender and receiver should be of the same type and both of same size at least *datasize*.

In most cases the *sendtype* and *recvtype* are identical.

The tag can be any integer between 0-32767.

MPI Recv may use for the tag the wildcard *MPI ANY TAG*. This allows an MPI Recv to receive from a send using any tag.
MPI Send cannot use the wildcard *MPI ANY TAG*. A special tag must be specified.
MPI Recv may use for the source the wildcard *MPI ANY SOURCE*. This allows an MPI Recv to receive from a send from any source.
MPI Send must specify the process rank of the destination. No wildcard exists.

Compile and Run the program

mpiexec -n 4 python **your_mpi_program**.py
This command will run your Python MPI program with 4 processes. Adjust the **-n** flag to specify the number of processes you want to use.

Activity 2:

An Example Program: To calculate the sum of given numbers in parallel:

The following program calculates the sum of numbers from 1 to 1000 in a parallel fashion while executing on all the cluster nodes and providing the result at the end on only one node. It should be noted that the print statement for the sum is only executed on the node that is ranked zero (0) otherwise the statement would be printed as much time as the number of nodes in the cluster.

```
#include<iostream.h>
#include<mpi.h>

int main(int argc, char ** argv)
{ int mynode, totalnodes; int sum,startval,endval,accum;
MPI_Status status;
    MPI_Init(argc,argv);

MPI_Comm_size(MPI_COMM_WORLD, &totalnodes);
MPI_Comm_rank(MPI_COMM_WORLD, &mynode);            sum = 0;
        startval = 1000*mynode/totalnodes+1;            endval = 1000*(mynode+1)/totalnodes;            for(int
i=startval;i<=endval;i=i+1)          sum = sum + i;          if(mynode!=0)
        MPI_Send(&sum,1,MPI_INT,0,1,MPI_COMM_WORLD);          else
        for(int j=1;j<totalnodes;j=j+1)
        {
MPI_Recv(&accum,1,MPI_INT,j,1,MPI_COMM_WORLD,
&status);
        sum = sum + accum;
        }
        if(mynode == 0) cout << "The sum from 1 to 1000 is: " <<  sum <<   endl;
        MPI_Finalize();
}
```

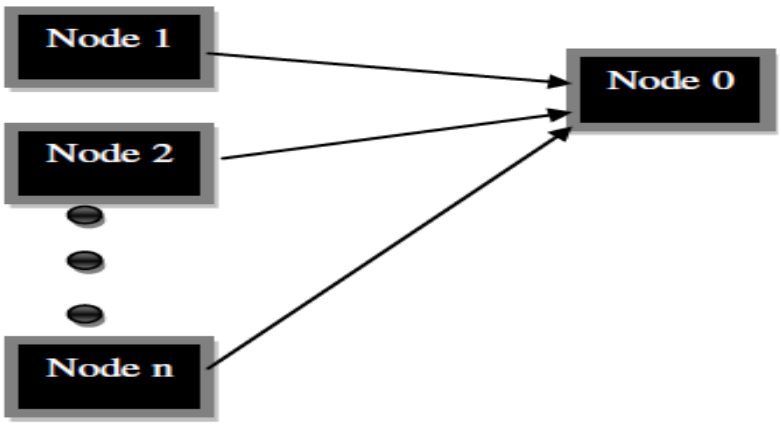
3) Lab Graded Tasks:

1) *Code the above example program in C that calculates the sum of numbers in parallel on different numbers of nodes. Also calculate the execution time.*

[Note: You have to use time stamp function to also print the time at begging and end of parallel code segment]

Output: (On Single Node) Execution Time: Output: (On Two Nodes) Execution Time: Speedup: Output: (On Four Nodes) Execution Time: Speedup: Output: (On Sixteen Nodes) Execution Time: Speedup:

2**.** *Suppose you are in a scenario where you have to transmit an array buffer from all other nodes to one node by using send/ receive functions that are used for intra-process synchronous communication. The figure below demonstrates the required functionality of the program.*


Node 1
Node 2
Node n
Node 0

LAB 08: Advanced MPI processes Communication (a)

Objective:

The objective of this lab is to teach the students about the Familiarized with advanced communication between MPI processes with the help of examples and learning tasks.

Activity:

In this lab, students will learn:

- Communication between MPI processes
- Simultaneous Send and Receive, MPI_Sendrecv

1) Useful Concepts

This Lab session will focus on more information about sending and receiving in MPI like sending of arrays and simultaneous send and receive

Key Points

- Whenever you send and receive data, MPI assumes that you have provided non overlapping positions in memory. As discussed in the previous lab session, MPI_COMM_WORLD is referred to as a communicator. In general, a communicator is a collection of processes that can send messages to each other. MPI_COMM_WORLD is pre-defined in all implementations of MPI, and it consists of all MPI processes running after the initial execution of the program.

- In the send/receive, we are required to use a tag. The tag variable is used to distinguish upon receipt between two messages sent by the same process.

- The order of sending does not necessarily guarantee the order of receiving. Tags are used to distinguish between messages. MPI allows the tag MPI_ANY_TAG which can be used by MPI_Recv to accept any valid tag from a sender but you cannot use MPI_ANY_TAG in the MPI_Send command.

- Similar to the MPI_ ANY_ TAG wildcard for tags, there is also an MPI_ANY_SOURCE wildcard that can also be used by MPI_Recv. By using it in an MPI_Recv, a process is ready to receive from any sending process. Again, you cannot use MPI_ ANY_ SOURCE in the MPI_ Send command. There is no wildcard for sender destinations.

- When you pass an array to MPI_ Send/MPI_Recv, it need not have exactly the number of items to be sent – it must have greater than or equal to the number of items to be sent. Suppose, for example, that you had an array of 100 items, but you only wanted to send the first ten items, you can do so by passing the array to MPI_Send and only stating that ten items are to be sent.

2) Solved Lab Activities

Sr.No	*Allocated Time*	*Level of Complexity*	*CLO Mapping*
1	*30*	*Low*	*CLO-4*

Activity 1:
An Example Program:

In the following MPI code, array on each process is created, initialize it on process 0. Once the array has been initialized on process 0, then it is sent out to each process.

```
#include<iostream.h>
#include<mpi.h>

int main(int argc, char * argv[])
{ int i; int nitems = 10; int mynode, totalnodes; MPI_Status status; double * array;

MPI_Init(&argc,&argv);
        MPI_Comm_size(MPI_COMM_WORLD,                                   &totalnodes);
MPI_Comm_rank(MPI_COMM_WORLD, &mynode);          array = new double[nitems];
        if(mynode == 0)
        {
        for(i=0;i<nitems;i++)          array[i] = (double) i;
        }          if(mynode==0)
        for(i=1;i<totalnodes;i++)
MPI_Send(array,nitems,MPI_DOUBLE,i,1,MPI_COMM_WORLD); else
MPI_Recv(array,nitems,MPI_DOUBLE,0,1,MPI_COMM_WORLD,
&status);
for(i=0;i<nitems;i++)
  {
   cout << "Processor " << mynode;
   cout << ": array[" << i << "] = " << array[i] <<  endl;
  }
MPI_Finalize();
}
```

Key Points:

- An array is created, on each process, using dynamic memory allocation.
- On process 0 only (i.e., mynode == 0), an array is initialized to contain the ascending index values.
- On process 0, program proceeds with (totalnodes-1) calls to MPI Send.
- On all other processes other than 0, MPI_Recv is called to receive the sent message.
- On each individual process, the results are printed of the sending/receiving pair.

Simultaneous Send and Receive, MPI_Sendrecv:

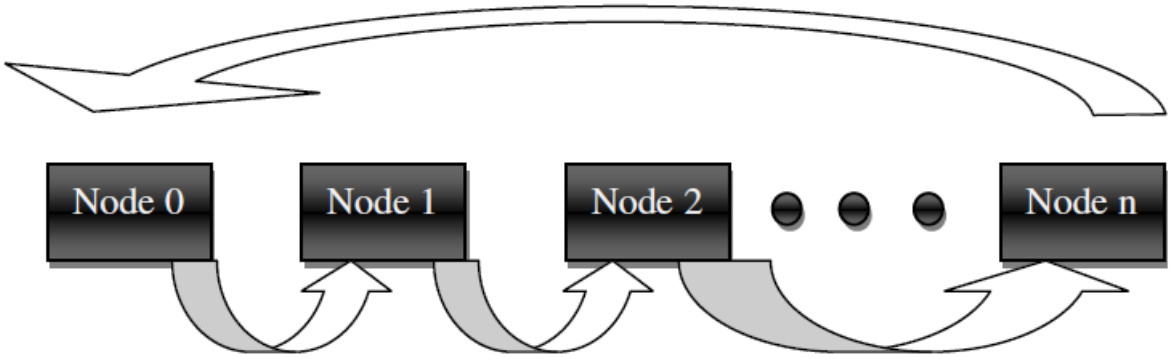
The subroutine MPI_Sendrecv exchanges messages with another process. A send-receive operation is useful for avoiding some kinds of unsafe interaction patterns and for implementing remote procedure calls. A message sent by a send-receive operation can be received by MPI_Recv and a send-receive operation can receive a message sent by an MPI_Send.

MPI_Sendrecv(&data_to_send, send_count, send_type, destination_ID, send_tag, &received_data, receive_count, receive_type, sender_ID, receive_tag, comm, &status)
Understanding the Argument Lists

- **data_to_send**: variable of a C type that corresponds to the MPI send_type supplied below
- **send_count:** number of data elements to send (int)
- **send_type:** datatype of elements to send (one of the MPI datatype handles)
- **destination_ID**: process ID of the destination (int)
- **send_tag**: send tag (int)
- **received_data**: variable of a C type that corresponds to the MPI receive_type supplied below
- **receive_count**: number of data elements to receive (int)
- **receive_type**: datatype of elements to receive (one of the MPI datatype handles)
- **sender_ID:** process ID of the sender (int)
- **receive_tag:** receive tag (int)
- **comm:** communicator (handle)
- **status:** status object (MPI_Status)

3) Lab Graded Tasks:

1. *Write a program in which every node receives from its left node and sends message to its right node simultaneously as depicted in the following figure*



2. Write a program to calculate prefix sum S_n of 'n' numbers on 'n' processes

- *Each node has two variables 'a' and 'b'.*
- *Initially a=node id*
- *Each node sends 'a' to the other node.*
- *'b' is a variable that receives a sent from another node.*

3. Write a parallel program that calculates the sum of array and execute it for different numbers of nodes in the cluster. Also calculate their respective execution time.

Output: (On Single Node) Execution Time: Output: (On Two Nodes) Execution Time: Speedup: Output: (On Four Nodes) Execution Time: Speedup: Output: (On Sixteen Nodes) Execution Time: Speedup:

LAB 09: Advanced MPI processes Communication (b)

Objective:

The objective of this lab is to teach the students about Study of Communication between MPI processes with the help of examples and learning tasks.

Activity Outcome:

In this lab, you learn about:

- Communication between MPI processes

1) Useful Concepts

It is important to observe that when a program running with MPI, all processes use the same compiled binary, and hence all processes are running the exact same code. What in an MPI distinguishes a parallel program running on P processors from the serial version of the code running on P processors? Two things distinguish the parallel program:

- Each process uses its process rank to determine what part of the algorithm instructions are meant for it.
- Processes communicate with each other in order to accomplish the final task.

Even though each process receives an identical copy of the instructions to be executed, this does not imply that all processes will execute the same instructions. Because each process is able to obtain its process rank (using MPI_Comm_rank).It can determine which part of the code it is supposed to run. This is accomplished through the use of IF statements. Code that is meant to be run by one particular process should be enclosed within an IF statement, which verifies the process identification number of the process. If the code is not placed with in IF statements specific to a particular id, then the code will be executed by all processes.

The second point, communicating between processes; MPI communication can be summed up in the concept of sending and receiving messages. Sending and receiving is done with the following two functions: MPI Send and MPI Recv.

- **MPI_Send C**

```
int MPI_Send( void* message /* in */, int count /* in */, MPI
Datatype datatype /* in */, int dest /* in */, int tag /* in
*/, MPI Comm comm /* in */ )
```

MPI Send Python

```python
def MPI_Send(message, count, datatype, dest, tag, comm):
    comm.send(message, dest=dest, tag=tag)

# Example usage:
comm = MPI.COMM_WORLD
rank = comm.Get_rank()

if rank == 0:
    data = "Hello, world!"
    dest_rank = 1
    tag = 123
    MPI_Send(data, len(data), MPI.CHAR, dest_rank, tag, comm)
```

- **MPI_Recv**

int MPI_Recv(void* message /* out */, int count /* in */, MPI Datatype datatype /* in */, int source /* in */, int tag /* in */, MPI Comm comm /* in */, MPI Status* status /* out */)

Understanding the Argument Lists

- **message** - starting address of the send/recv buffer.
- **count** - number of elements in the send/recv buffer.
- **datatype** - data type of the elements in the send buffer.
- **source** - process rank to send the data. **dest -** process rank to receive the data.
- **tag** - message tag.
- **comm** - communicator.
- **status** - status object.

MPI Programming using Python.

An Example Program:

MPI (Message Passing Interface) is a popular parallel computing framework used for distributed-memory systems. In Python, you can use the mpi4py library to write MPI programs should be initialized to suitable ranks.

Installation

pip install mpi4py

```
from mpi4py import MPI

def calculate_sum(rank, size, n):
    local_sum = 0
    for i in range(rank, n, size):
        local_sum += i
    return local_sum

def main():
    comm = MPI.COMM_WORLD
    rank = comm.Get_rank()
    size = comm.Get_size()

    n = 1000  # Example: calculating sum of numbers from 0 to 999
```

```
    local_sum = calculate_sum(rank, size, n)
    total_sum = comm.reduce(local_sum, op=MPI.SUM, root=0)

    if rank == 0:
        print("Total sum:", total_sum)

if __name__ == "__main__":
    main()
```

Explanation:

- We import MPI from the mpi4py library.
- We define a function calculate_sum that calculates the sum of numbers from rank to n-1 with a given stride size.
- In the main function, we initialize MPI and get the rank and size of the communicator.
- We define the value of n, which is the upper bound for the sum.
- Each process calculates its local sum.
- We use comm.reduce to gather all the local sums to the root process (rank 0) and perform a reduction operation (sum in this case).
- Finally, the root process prints out the total sum.

MPI Programming using C

```
#include <iostream.h>
#include <mpi.h>

int main(int argc, char ** argv
{ int mynode, totalnodes;
int datasize; // number of data units to be sent/recv int sender=2; // process number of the sending process int receiver=4; // process number of the receiving process int tag; // integer message tag
MPI_Status status; // variable to contain status information

MPI_Init(&argc,&argv);

MPI_Comm_size(MPI_COMM_WORLD, &totalnodes);
MPI_Comm_rank(MPI_COMM_WORLD, &mynode);

// Determine datasize
double * databuffer = new double[datasize];
// Fill in sender, receiver, tag on sender/receiver processes,
// and fill in databuffer on the sender process.

if(mynode==sender)

MPI_Send(databuffer,datasize,MPI_DOUBLE,receiver,
tag,MPI_COMM_WORLD);

if(mynode==receiver)
    MPI_Recv(databuffer,datasize,MPI_DOUBLE,sender,tag,
MPI_COMM_WORLD,&status);
// Send/Recv complete
  MPI_Finalize();
}
```

Key Points:

- In general, the *message* array for both the sender and receiver should be of the same type and both of same size at least *datasize*.
- In most cases the *sendtype* and *recvtype* are identical.
- The tag can be any integer between 0-32767.
- *MPI Recv* may use for the tag the wildcard *MPI ANY TAG*. This allows an MPI Recv to receive from a send using any tag.
- *MPI Send* cannot use the wildcard *MPI ANY TAG*. A special tag must be specified.
- *MPI Recv* may use for the source the wildcard *MPI ANY SOURCE*. This allows an MPI Recv to receive from a send from any source.
- MPI Send must specify the process rank of the destination. No wildcard exists.

2) Solved Lab Activities

Sr.No	***Allocated Time***	***Level of Complexity***	***CLO Mapping***
1	***30***	***Low***	***CLO-4***

Activity 1:

An Example Program Using C: To calculate the sum of given numbers in parallel:

The following program calculates the sum of numbers from 1 to 1000 in a parallel fashion while executing on all the cluster nodes and providing the result at the end on only one node. It should be noted that the print statement for the sum is only executed on the node that is ranked zero (0) otherwise the statement would be printed as much time as the number of nodes in the cluster.

```
#include<iostream.h>
#include<mpi.h>

int main(int argc, char ** argv)
{ int mynode, totalnodes; int sum,startval,endval,accum;
MPI_Status status;
    MPI_Init(argc,argv);

MPI_Comm_size(MPI_COMM_WORLD, &totalnodes);
MPI_Comm_rank(MPI_COMM_WORLD, &mynode);          sum = 0;
        startval = 1000*mynode/totalnodes+1;          endval = 1000*(mynode+1)/totalnodes;          for(int
i=startval;i<=endval;i=i+1)          sum = sum + i;          if(mynode!=0)
        MPI_Send(&sum,1,MPI_INT,0,1,MPI_COMM_WORLD);          else
        for(int j=1;j<totalnodes;j=j+1)
        {
MPI_Recv(&accum,1,MPI_INT,j,1,MPI_COMM_WORLD,
&status);
        sum = sum + accum;
        }
        if(mynode == 0) cout << "The sum from 1 to 1000 is: " <<  sum <<   endl;
        MPI_Finalize();
}
```

Python Code

```
from mpi4py import MPI

def main():
    mynode = MPI.COMM_WORLD.Get_rank()
    totalnodes = MPI.COMM_WORLD.Get_size()
    sum_val = 0
    startval = 1000 * mynode // totalnodes + 1
    endval = 1000 * (mynode + 1) // totalnodes

    for i in range(startval, endval + 1):
        sum_val += i

    if mynode != 0:
        MPI.COMM_WORLD.send(sum_val, dest=0, tag=1)
    else:
        for j in range(1, totalnodes):
            accum = MPI.COMM_WORLD.recv(source=j, tag=1)
            sum_val += accum
```

```
    else:
        for j in range(1, totalnodes):
            accum = MPI.COMM_WORLD.recv(source=j, tag=1)
            sum_val += accum

        print("The sum from 1 to 1000 is:", sum_val)

if __name__ == "__main__":
    MPI.Init()
    main()
    MPI.Finalize()
```

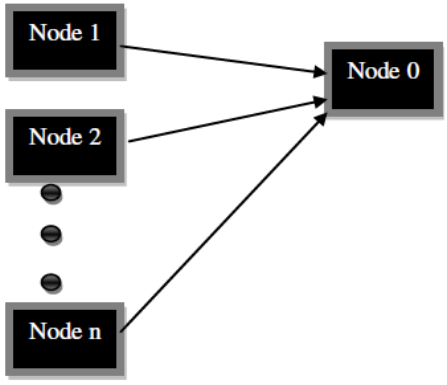
Remember, when running MPI programs, you typically use a command like mpirun or mpiexec to launch the program across multiple processes:

3) Lab Graded Tasks:

1. Code the above example program in C that calculates the sum of numbers in parallel on different numbers of nodes. Also calculate the execution time.
[Note: You have to use time stamp function to also print the time at begging and end of parallel code segment]

Output: (On Single Node)
Execution Time:
Output: (On Two Nodes)
Execution Time:
Speedup:
Output: (On Four Nodes)
Execution Time:
Speedup:
Output: (On Sixteen Nodes)
Execution Time:
Speedup:

2. Suppose you are in a scenario where you have to transmit an array buffer from all other nodes to one node by using send/ receive functions that are used for intra-process synchronous communication. The figure below demonstrates the required functionality of the program.



LAB 10: MPI collective operations using ‘Synchronization’

Objective:

The objective of this lab is to teach the students about Study of MPI collective operations using ‘Synchronization’ with the help of examples and learning tasks.

Activity Outcomes:

In this lab, Students will learn:

- Study of MPI collective operations using ‘Synchronization’

1) Useful Concepts

Collective operations

MPI_Send and MPI_Recv are "point-to-point" communications functions. That is, they involve one sender and one receiver. MPI includes a large number of subroutines for performing "collective" operations. Collective operations are performed by MPI routines that are called by each member of a group of processes that want some operation to be performed for them as a group. A collective function may specify one-to-many, many-to-one, or manyto-many message transmission. MPI supports three classes of collective operations:

- Synchronization,
- Data Movement, and
- Collective Computation

These classes are not mutually exclusive, of course, since blocking data movement functions also serve to synchronize process activity, and some MPI routines perform both data movement and computation.

Synchronization

The MPI_Barrier function can be used to synchronize a group of processes. To synchronize a group of processes, each one must call MPI_Barrier when it has reached a point where it can go no further until it knows that all its partners have reached the same point. Once a process has called MPI_Barrier, it will be blocked until all processes in the group have also called MPI_Barrier.

MPI Barrier

int MPI_Barrier(MPI Comm comm /* in */)

Understanding the Argument Lists

- comm - communicator

2) Solved Lab Activities

Sr.No	*Allocated Time*	*Level of Complexity*	*CLO Mapping*
1	**30**	**Complex**	*CLO-4*

Activity 1:

Example Usage:

```
int mynode, totalnodes;

MPI_Init(&argc,&argv);
MPI_Comm_size(MPI_COMM_WORLD, &totalnodes);
MPI_Comm_rank(MPI_COMM_WORLD, &mynode);

MPI_Barrier(MPI_COMM_WORLD);
// At this stage, all processes are synchronized
```

Key Point

This command is a useful tool to help insure synchronization between processes. For example, you may want all processes to wait until one particular process has read in data from disk. Each process would call *MPI_Barrier* in the place in the program where the synchronization is required.

3) Graded Lab Activities

1. Write a parallel program, after discussing your instructor which uses MPI_Barrier

LAB 11 : MPI collective operations using ‘Data Movement’

Objective:

The objective of this lab is to teach the students about : Study of MPI collective operations using ‘Data Movement’ with the help of examples and learning tasks.

Activity Outcomes:

In this lab, will learn:

MPI collective operations using ‘Data Movement’

1) Useful Concepts

Collective operations

MPI_Send and MPI_Recv are "point-to-point" communications functions. That is, they involve one sender and one receiver. MPI includes a large number of subroutines for performing "collective" operations. Collective operations are performed by MPI routines that are called by each member of a group of processes that want some operation to be performed for them as a group. A collective function may specify one-to-many, many-to-one, or manyto-many message transmission. MPI supports three classes of collective operations:

- Synchronization,
- Data Movement, and
- Collective Computation

These classes are not mutually exclusive, of course, since blocking data movement functions also serve to synchronize process activity, and some MPI routines perform both data movement and computation.

Collective data movement

MPI_Bcast The subroutine MPI_Bcast sends a message from one process to all processes in a communicator.

MPI_Gather, MPI_Gatherv, Gather data from participating processes into a single structure

MPI_Scatter, MPI_Scatterv, Break a structure into portions and distribute those portions to other processes

MPI_Allgather, MPI_Allgatherv, Gather data from different processes into a single structure that is then sent to all participants (Gather-to-all)

MPI_Alltoall, MPI_Alltoallv, Gather data and then scatter it to all participants (Allto-all scatter/gather)

The routines with "V" suffixes move variable-sized blocks of data.

MPI_Bcast

- The subroutine MPI_Bcast sends a message from one process to all processes in a communicator.
- In a program all processes must execute a call to MPI_BCAST. There is no separate MPI call to receive a broadcast

buffer - starting address of the send buffer.

- **count** - number of elements in the send buffer.

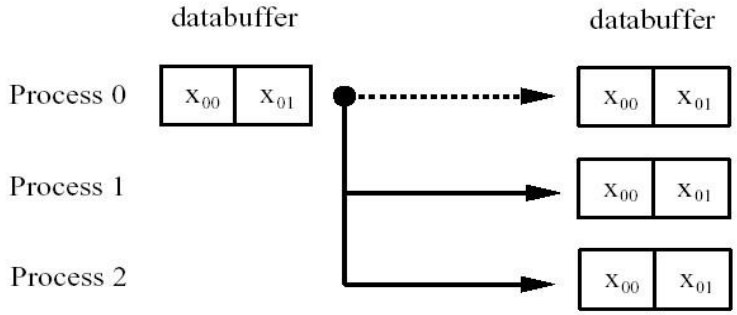


Figure *MPI Bcast* schematic demonstrating a broadcast of two data objects from process zero to all other processes.

```
int MPI Bcast( void* buffer /* in/out */, int count /* in
MPI Datatype datatype /* in */,
int root /* in */, MPI Comm comm /* in */)
```

Understanding the Argument List

- **datatype** - data type of the elements in the send buffer.
- **root** - rank of the process broadcasting its data.
- **comm** - communicator.

MPI_Bcast broadcasts a message from the process with rank "root" to all other processes of the group.

2) Solved Lab Activities

Sr.No	*Allocated Time*	*Level of Complexity*	*CLO Mapping*
1	*30*	*Low*	*CLO-4*

Activity 1:

Example Usage

int mynode, totalnodes;
int datasize; // number of data units to be broadcast int root; // process which is broadcasting its data

MPI_Init(&argc,&argv);
MPI_Comm_size(MPI_COMM_WORLD, &totalnodes);
MPI_Comm_rank(MPI_COMM_WORLD, &mynode);

// Determine datasize and root
double * databuffer = new double[datasize];
// Fill in databuffer array with data to be broadcast

MPI_Bcast(databuffer,datasize,MPI_DOUBLE,root,MPI_COMM_WORLD);

// At this point, every process has received into the
// databuffer array the data from process root

Key Point

Each process will make an identical call of the *MPI Bcast* function. On the broadcasting (root) process, the *buffer* array contains the data to be broadcast. At the conclusion of the call, all processes have obtained a copy of the contents of the *buffer* array from process root.

MPI_Scatter:

MPI_Scatter is one of the most frequently used functions of MPI Programming. Break a structure into portions and distribute those portions to other processes. Suppose you are going to distribute an array elements equally to all other nodes in the cluster by decomposing the main array into its sub segments which are then distributed to the nodes for parallel computation of array segments on different cluster nodes.

int MPI_Scatter
(void *send_data, int send_count, MPI_Datatype send_type, void *receive_data, int receive_count, MPI_Datatype receive_type, int sending_process_ID,
MPI_Comm comm.
)

MPI_Gather

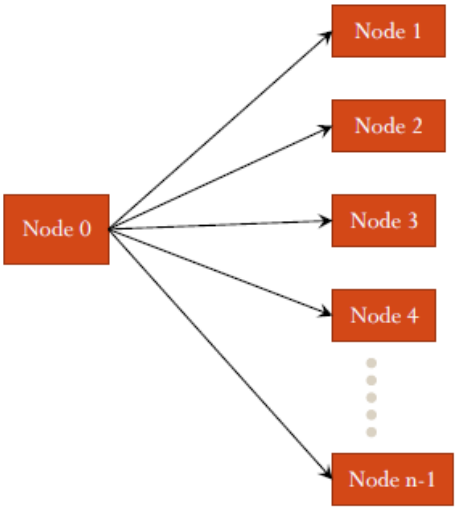
Gather data from participating processes into a single structure Synopsis: #include "mpi.h"
int MPI_Gather
void *sendbuf, int sendcnt,
MPI_Datatype sendtype, void *recvbuf, int recvcount, MPI_Datatype recvtype, int root, MPI_Comm comm
)
Input Parameters:

sendbuf: starting address of send buffer
sendcount: number of elements in send buffer
sendtype: data type of send buffer elements
recvcount: number of elements for any single receive (significant only at root)
recvtype: data type of recv buffer elements (significant only at root)
root: rank of receiving process
comm: communicator

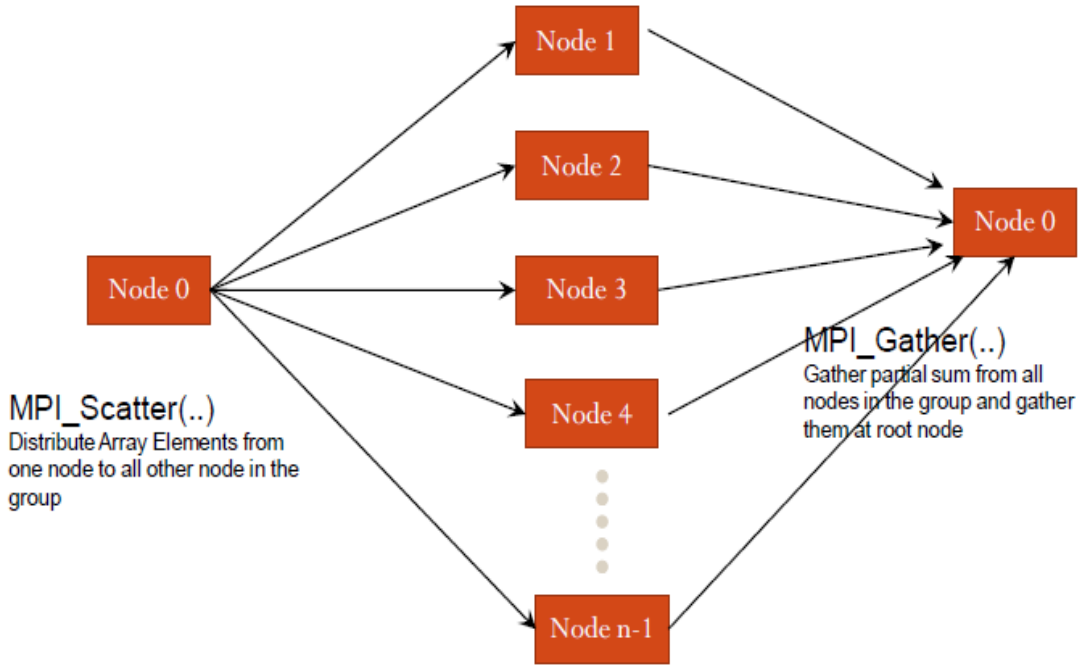
Output Parameter:
recvbuf: address of receive buffer (significant only at root)

3) Lab Graded Tasks:

1. *Write a program that broadcasts a number from one process to all others by using MPI_Bcast.*



2. Break up a long vector into sub-vectors of equal length. Distribute sub-vectors to processes. Let the processes to compute the partial sums. Collect the partial sums from the processes and add them at root node using collective computation operations



3. Write a parallel program that calculates the value of PI using integral method.

Algorithm: The algorithm suggested here is chosen for its simplicity. The method evaluates the integral of 4/(1+x*x) between -1/2 and 1/2. The method is simple: the integral is approximated by a sum of n intervals; the approximation to the integral in each interval is (1/n)*4/(1+x*x). The master process (rank 0) asks the user for the number of intervals; the master should then broadcast this number to all of the other processes. Each process then adds up every n'th interval (x = -1/2+rank/n, -1/2+rank/n+size/n,). Finally, the sums computed by each process are added together using a reduction.

LAB 12: MPI collective operations using ‘Collective Computation

Objective:

The objective of this lab is to teach the students about Study of MPI collective operations using ‘Collective Computation’ with the help of examples and learning tasks.

Activity Outcome:

In this lab, students will learn:

- Collective operations
- Collective Computation Routines
- Collective computation built-in operations
- MPI_Reduce

1) Useful Concepts

Collective operations

MPI_Send and MPI_Recv are "point-to-point" communications functions. That is, they involve one sender and one receiver. MPI includes a large number of subroutines for performing "collective" operations. Collective operations are performed by MPI routines that are called by each member of a group of processes that want some operation to be performed for them as a group. A collective function may specify one-to-many, many-to-one, or manyto-many message transmission. MPI supports three classes of collective operations:

- Synchronization,
- Data Movement, and
- Collective Computation

These classes are not mutually exclusive, of course, since blocking data movement functions also serve to synchronize process activity, and some MPI routines perform both data movement and computation.

Collective Computation Routines

Collective computation is similar to collective data movement with the additional feature that data may be modified as it is moved. The following routines can be used for collective computation.

MPI_Reduce: Perform a reduction operation.

MPI_Allreduce: Perform a reduction leaving the result in all participating processes

MPI_Reduce_scatter: Perform a reduction and then scatter the result

MPI_Scan: Perform a reduction leaving partial results (computed up to the point of a process's involvement in the reduction tree traversal) in each participating process. (parallel prefix)

Collective computation built-in operations

Many of the MPI collective computation routines take both built-in and user-defined combination functions. The built-in functions are:

Table 2. Collective Computation Operations

Operation handle	**Operation**
MPI_MAX	Maximum
MPI_MIN	Minimum
MPI_PROD	Product
MPI_SUM	Sum
MPI_LAND	Logical AND
MPI_LOR	Logical OR

MPI_LXOR	Logical Exclusive OR
MPI_BAND	Bitwise AND
MPI_BOR	Bitwise OR
MPI_BXOR	Bitwise Exclusive OR
MPI_MAXLOC	Maximum value and location
MPI_MINLOC	Minimum value and location

MPI_Reduce:

MPI_Reduce apply some operation to some operand in every participating process. For example, add an integer residing in every process together and put the result in a process specified in the MPI_Reduce argument list. The subroutine MPI_Reduce combines data from all processes in a communicator using one of several reduction operations to produce a single result that appears in a specified target process.

When processes are ready to share information with other processes as part of a data reduction, all of the participating processes execute a call to MPI_Reduce, which uses local data to calculate each process's portion of the reduction operation and communicates the local result to other processes as necessary. Only the target_process_ID receives the final result.

int MPI_Reduce(void* operand /* in */, void* result /* out */, int count /* in */,
MPI Datatype datatype /* in */, MPI Op operator /* in */, int root /* in */, MPI Comm comm /* in */
)

Understanding the Argument List

operand - starting address of the send bu.er.
result - starting address of the receive bu.er.
count - number of elements in the send bu.er.
datatype - data type of the elements in the send/receive bu.er.
operator - reduction operation to be executed.
root - rank of the root process obtaining the result.
comm - communicator.

2) Solved Lab Activities

Sr.No	*Allocated Time*	*Level of Complexity*	*CLO Mapping*
1	*25*	*Complex*	*CLO-4*
2	*30*	*Complex*	*CLO-4*

Activity 1:

Example Usage:

The given code receives data on only the root node (rank=0) and passes null in the receive data argument of all other nodes

```
int mynode, totalnodes;
int datasize; // number of data units over which
// reduction should occur
int root; // process to which reduction will occur

MPI_Init(&argc,&argv);
MPI_Comm_size(MPI_COMM_WORLD, &totalnodes);
MPI_Comm_rank(MPI_COMM_WORLD, &mynode);

// Determine datasize and root double * senddata = new double[datasize]; double * recvdata = NULL;

if(mynode == root)
recvdata = new double[datasize]; // Fill in senddata on all processes

MPI_Reduce(senddata,recvdata,datasize,MPI_DOUBLE,MPI_SUM, root,MPI_COMM_WORLD);

// At this stage, the process root contains the result of the reduction (in this    case MPI_SUM) in the recvdata array
```

Key Points

The recvdata array only needs to be allocated on the process of rank root (since root is the only processor receiving data). All other processes may pass NULL in the place of the recvdata argument.

Both the senddata array and the recvdata array must be of the same data type. Both arrays should contain at least datasize elements.

MPI_Allreduce*:* Perform a reduction leaving the result in all participating processes

```
int MPI Allreduce( void* operand /* in */, void* result /* out */, int count /* in */,
MPI Datatype datatype /* in */,
MPI Op operator /* in */,
MPI Comm comm /* in */
)
```

Understanding the Argument List
operand - starting address of the send buffer.
result - starting address of the receive buffer.
count - number of elements in the send/receive buffer.
datatype - data type of the elements in the send/receive buffer.
operator - reduction operation to be executed.
comm - communicator.

Activity 2:

Example Usage:

int mynode, totalnodes;

```
int datasize; // number of data units over which
// reduction should occur

MPI_Init(&argc,&argv);
MPI_Comm_size(MPI_COMM_WORLD, &totalnodes);
MPI_Comm_rank(MPI_COMM_WORLD, &mynode);

// Determine datasize and root double * senddata = new double[datasize]; double * recvdata = new double[datasize];
// Fill in senddata on all processes

MPI_Allreduce(senddata,recvdata,datasize,MPI_DOUBLE,
MPI_SUM,MPI_COMM_WORLD);
// At this stage, all processes contains the result  of the reduction (in this case MPI_SUM) in the recvdata array
```

Remarks

In this case, the *recvdata* array needs to be allocated on all processes since all processes will be receiving the result of the reduction.

Both the *senddata* array and the *recvdata* array must be of the same data type. Both arrays should contain at least *datasize* elements.

MPI_Scan:

MPI_Scan: Computes the scan (partial reductions) of data on a collection of processes
Synopsis:

```
#include "mpi.h"
int MPI_Scan (void *sendbuf, void *recvbuf, int count, MPI_Datatype datatype, MPI_Op op, MPI_Comm comm)
```

- Input Parameters
- sendbuf: starting address of send buffer
- count: number of elements in input buffer

- datatype: data type of elements of input buffer
- op: operation
- comm: communicator

- Output Parameter:

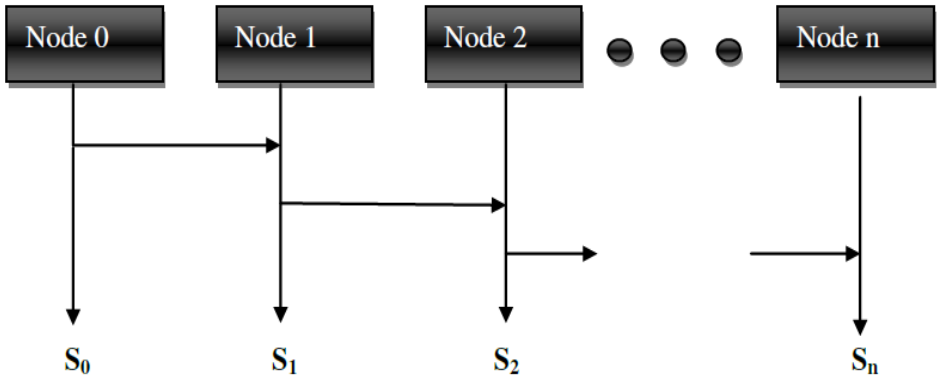
recvbuf: starting address of receive buffer

3) Lab Graded Tasks:

1. Break up a long vector into sub-vectors of equal length. Distribute sub-vectors to processes. Let the processes to compute the partial sums. Collect the partial sums from the processes and add them at root node using collective computation operations

2. Write a program to calculate prefix sum of 'n' numbers on 'n' processes. Use MPI_Scan to address this problem.

The above problem can be best stated with the help of following figure.



3. Write and explain argument lists of the following and say how they are different from the two functions you have seen:

- MPI_Allreduce
- MPI_Reduce_scatter

LAB # 13: MPI Non-Blocking operation

Objective:

The objective of this lab is to teach the students about To understand MPI Non-Blocking operation with the help of examples and learning tasks.

Activity Outcome:

In this lab, students will learn about:
MPI: Non-Blocking Communications
Why Non Blocking Communication?
Understanding the Argument Lists

1) Useful Concepts

MPI: Non-Blocking Communications

Non-blocking point-to-point operation allows overlapping of communication and computation to use the common parallelism in modern computer systems more efficiently. This enables the user to use the CPU even during ongoing message transmissions at the network level.

As *MPI_Send and MPI_Recv* or *MPI Sendrecv fuctions* require some level of synchronization for associating the corresponding sends and receives on the appropriate processes. *MPI_Send* and *MPI_Recv* are *blocking communications*, which means that they will not return until it is safe to modify or use the contents of the send/recv buffer respectively.

MPI also provides *non-blocking versions* of these functions called *MPI_Isend* and *MPI_Irecv*, where the "I" stands for immediate. These functions allow a process to post that it wants to send to or receive from a process, and then later is allowed to call a function (*MPI Wait*) to complete the sending/receiving. These functions are useful in that they allow the programmer to appropriately stagger computation and communication to minimize the total waiting time due to communication.

The understand the basic idea behind MPI_Isend and MPI_Irecv, Suppose process 0 needs to send information to process 1, but due to the particular algorithms that these two processes are running, the programmer knows that there will be a mismatch in the synchronization of these processes. Process 0 initiates an MPI_Isend to process 1 (posting that it wants to send a message), and then continues to accomplish things which do not require the contents of the buffer to be sent. At the point in the algorithm where process 0 can no longer continue without being guaranteed that the contents of the sending buffer can be modified, process 0 calls MPI Wait to wait until the transaction is completed. On process 1, a similar situation occurs, with process 1 posting via MPI_Irecv that it is willing to accept a message. When process 1 can no longer continue without having the contents of the receive buffer, it too calls MPI_Wait to wait until the transaction is complete. At the conclusion of the MPI_Wait, the sender may modify the send buffer without compromising the send, and the receiver may use the data contained within the receive buffer.

Why Non-Blocking Communication?

The communication can consume a huge part of the running time of a parallel application. The communication time in those applications can be addressed as overhead because it does not progress the solution of the problem in most cases (with exception of reduce operations). Using overlapping techniques enables the user to move communication and the necessary synchronization in the background and use parts of the original communication time to perform useful computation.

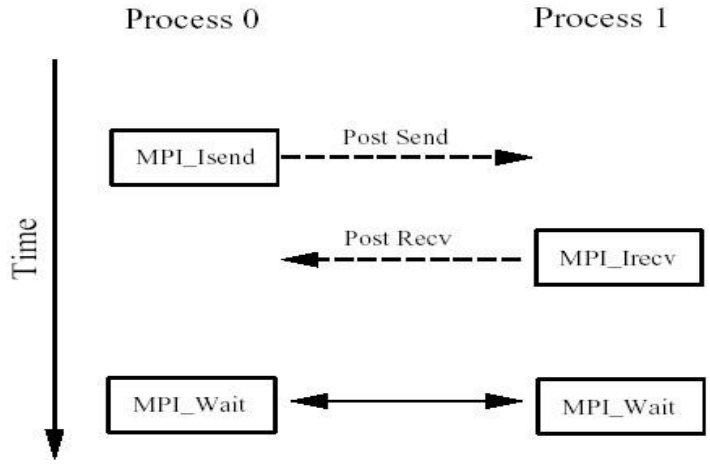


Figure7.1 MPI Isend/MPI Irecv schematic demonstrating the communication between two processes.

Function Call Syntax

```
int MPI Isend ( void* message /* in */, int count /* in */,
MPI Datatype datatype /* in */, int dest /* in */, int tag /* in */, MPI Comm comm /* in */,
MPI Request* request /* out */
)
```

```
int MPI Irecv( void* message /* out */, int count /* in */,
MPI Datatype datatype /* in */, int source /* in */, int tag /* in */, MPI Comm comm /* in */,
MPI Request* request /* out */
)
```

```
int MPI Wait(
MPI Request* request /* in/out */
MPI Status* status /* out */
)
```

Understanding the Argument Lists

message - starting address of the send/recv bu.er.
count - number of elements in the send/recv bu.er.
datatype - data type of the elements in the send bu.er.
source - process rank to send the data.
dest - process rank to receive the data.
tag - message tag.
comm - communicator.
request - communication request.
status - status object.

2) Solved Lab Activities

Sr.No	*Allocated Time*	*Level of Complexity*	*CLO Mapping*
1	*25*	*Low*	*CLO-4*

Example Usage:

```
int mynode, totalnodes;
int datasize; // number of data units to be sent/recv int sender; // process number of the sending process int receiver; // process number of the receiving process int tag; // integer message tag
MPI_Status status; // variable to contain status information
MPI_Request request; // variable to maintain
// isend/irecv information

MPI_Init(&argc,&argv);
MPI_Comm_size(MPI_COMM_WORLD, &totalnodes);
MPI_Comm_rank(MPI_COMM_WORLD, &mynode);

// Determine datasize
double * databuffer = new double[datasize]; // Fill in sender, receiver, tag on sender/receiver processes,
// and fill in databuffer on the sender process.

if(mynode==sender)
MPI_Isend(databuffer,datasize,MPI_DOUBLE,receiver,tag,
MPI_COMM_WORLD,&request);

if(mynode==receiver)
MPI_Irecv(databuffer,datasize,MPI_DOUBLE,sender,tag,
MPI_COMM_WORLD,&request);
// The sender/receiver can be accomplishing various things
// which do not involve the databuffer array

MPI_Wait(&request,&status); //synchronize to verify
// that data is sent
// Send/Recv complete
```

Key Points

In general, the *message* array for both the sender and receiver should be of the same type and both of size at least *datasize*.

In most cases the *sendtype* and *recvtype* are identical.

After the *MPI_Isend* call and before the *MPI_Wait* call, the contents of *message* should not be changed.

After the *MPI_Irecv* call and before the *MPI_Wait* call, the contents of *message* should not be used.

An *MPI_Send* can be received by an *MPI_Irecv/MPI_Wait*.

An *MPI_Recv* can obtain information from an *MPI_Isend/MPI_Wait*.

The tag can be any integer between 0-32767.
MPI_Irecv may use for the tag the wildcard *MPI_ANY_TAG*. This allows an *MPI_Irecv* to receive from a send using any tag.
MPI_Isend cannot use the wildcard *MPI_ANY_TAG*. A specific tag must be specified.
MPI_Irecv may use for the source the wildcard *MPI_ANY_SOURCE*. This allows an *MPI_Irecv* to receive from a send from any source.
MPI_Isend must specify the process rank of the destination. No wildcard exists.

3) Lab Graded Tasks:

1. Write a parallel program having the non-blocking processes communications which calculates the sum of numbers in parallel on different numbers of nodes. Also calculate the execution time.

Output: (On Single Node) Execution Time: Output: (On Two Nodes) Execution Time: Speedup: Output: (On Four Nodes) Execution Time: Speedup: Output: (On Sixteen Nodes) Execution Time: Speedup:

2. Write two programs that utilizes the functions MPI_Waitall and MPI_Waitany respectively.

LAB 14: Introduction to GPU programming with Numba

Objective:

The objective of this lab is to teach the students about Study of Introduction to GPU programming with Numba with the help of examples and learning tasks.

Activity Outcome:

In this lab, students will learn:

- Accessing the GPU
- Vector Addition on GPUs
- Memory Management

1) Useful Concepts

"Numba is a just-in-time compiler for Python that works best on code that uses NumPy arrays and functions, and loops. For our purposes today, Numba is a python package that allows you to write python code for GPUs.

```
%matplotlib inline
from matplotlib import pyplot as plt
import numpy as np
import math
from numba import jit, njit, vectorize, cuda
```

Accessing the GPU

- In order to run Numba functions using google's free GPUs, we have to do a couple of things. First, go to the Runtime menu, click on 'Change Runtime Type', and in the pop-up box, under 'Hardware Accelerator', select 'GPU'. Save the Runtime.
- Ideally, that's all we should have to do. But in practice, even though the CUDA libararies are installed, as of this writing Colab can't find them automatically. So, we'll figure out where they are, and then point Colab to them.

```
!find / -iname 'libdevice'
!find / -iname 'libnvvm.so'
```

Paste the location of the libraries into the following code box (if it's different, otherwise you can just run the code):

```
import os
os.environ['NUMBAPRO_LIBDEVICE'] = "/usr/local/cuda-10.0/nvvm/libdevice"
os.environ['NUMBAPRO_NVVM'] = "/usr/local/cuda-10.0/nvvm/lib64/libnvvm.so"
```

Vector Addition on GPUs

The simplest way to access the GPU via Numba is to use a vectorized ufunc. A Numpy ufunc, or Universal Function, is a function that operates on vectors, or arrays. If we use Numba's vectorize decorator and specify the cuda target, Numba will automatically write a CUDA kernel for us and run the function on the GPU! Let's try it out:

```
@vectorize(['int64(int64, int64)'], target='cuda')
def add_ufunc_gpu(x, y):
    return x + y
x = np.arange(10)
y = 2 * x
add_ufunc_gpu(x, y)
```

Cool, it worked! But what actually just happened? Well, a lot of things. Numba automatically:

-Compiled a CUDA kernel to execute the ufunc operation in parallel over all the input elements.

-Allocated GPU memory for the inputs and the output.

-Copied the input data to the GPU.

-Executed the CUDA kernel with the correct kernel dimensions given the input sizes.

-Copied the result back from the GPU to the CPU.

-Returned the result as a NumPy array on the host.

Using the %timeit magic function, we can determine how fast the CUDA function is:

```
%timeit add_ufunc_gpu(x, y)
```

And compare it to a version compiled for the CPU:

```
@vectorize(['int64(int64, int64)'], target='cpu')
def add_ufunc_cpu(x, y):
   return x + y
%timeit add_ufunc_cpu(x, y)
```

Writing Cuda Kernels While targeting ufuncs with the cuda syntax is the most straightforward way to access the GPU with Numba, it may not be flexible enough for your needs. If you want to write a more detailed GPU program, at some point you are probably going to need to write CUDA kernels.

As discussed in the lecture, the CUDA programming model allows you to abstract the GPU hardware into a software model composed of a grid containing blocks of threads. These threads are the smallest individual unit in the programming model, and they execute together in groups (traditionally called warps, consisting of 32 threads each). Determiming the best size for your grid of thread blocks is a complicated problem that often depends on the specific algorithm and hardware you're using, but here a few good rules of thumb:

-the size of a block should be a multiple of 32 threads, with typical block sizes between 128 and 512 threads per block.

-the size of the grid should ensure the full GPU is utilized where possible. Launching a grid where the number of blocks is 2x-4x the number of streaming multiprocessors on the GPU is a good starting place. (The Tesla K80 GPUs provided by Colaboratory have 15 SMs - more modern GPUs like the P100s on TigerGPU have 60+.)

-The CUDA kernel launch overhead does depend on the number of blocks, so it may not be best to launch a grid where the number of threads equals the number of input elements when the input size is very big. We'll show a pattern for dealing with large inputs below.

As a first example, let's return to our vector addition function, but this time, we'll target it with the cuda.jit decorator:

```
@cuda.jit
def add_kernel(x, y, out):
   tidx = cuda.threadIdx.x # this is the unique thread ID within a 1D block
   bidx = cuda.blockIdx.x  # Similarly, this is the unique block ID within the 1D grid

   block_dimx = cuda.blockDim.x  # number of threads per block
   grid_dimx = cuda.gridDim.x    # number of blocks in the grid

   start = tidx + bidx * block_dimx
   stride = block_dimx * grid_dimx

   # assuming x and y inputs are same length
```

```
    for i in range(start, x.shape[0], stride):
        out[i] = x[i] + y[i]
```

That's a lot more typing than our ufunc example, and it is much more limited: it only works on 1D arrays, it doesn't verify input sizes match, etc. Most of the function is spent figuring out how to turn the block and grid indices and dimensions into unique offsets in the input arrays. The pattern of computing a starting index and a stride is a common way to ensure that your grid size is independent of the input size. The striding will maximize bandwidth by ensuring that threads with consecuitive indices are accessing consecutive memory locations as much as possible. Thread indices beyond the length of the input (x.shape[0], since x is a NumPy array) automatically skip over the for loop.

Let's call the function now on some data:

```
n = 100000
x = np.arange(n).astype(np.float32)
y = 2 * x
out = np.empty_like(x)

threads_per_block = 128
blocks_per_grid = 30

add_kernel[blocks_per_grid, threads_per_block](x, y, out)
print(out[:10])
```

The calling syntax is designed to mimic the way CUDA kernels are launched in C, where the number of blocks per grid and threads per block are specified in the square brackets, and the arguments to the function are specified afterwards in parentheses.
Note that, unlike the ufunc, the arguments are passed to the kernel as full NumPy arrays. A thread within the kernel can access any element in the array it wants, regardless of its position in the thread grid. This is why CUDA kernels are significantly more powerful than ufuncs. (But with great power, comes a greater amount of typing...)
Numba has created some helper functions to cut down on the typing. We can write the previous kernel much more simply as:

```
@cuda.jit
def add_kernel(x, y, out):
start = cuda.grid(1)
stride = cuda.gridsize(1) # ditto
for i in range(start, x.shape[0], stride):
out[i] = x[i] + y[i]
```

As before, using NumPy arrays forces Numba to allocate GPU memory, copy the arguments to the GPU, run the kernel, then copy the argument arrays back to the host. This not very efficient, so you will often want to pre-allocate device arrays.

Memory Management

Numba can automatically handle transferring data to and from the GPU for us. However, that's not always what we want. Sometimes we will want to perform several functions in a row on the GPU without transferring the data back to the CPU in between. To address this, Numba provides the to_device function in the cuda module to allocate and copy arrays to the GPU:

```
x_device = cuda.to_device(x)
y_device = cuda.to_device(y)
print(x_device)
print(x_device.shape)
print(x_device.dtype)
```

x_device and y_device are now Numba "device arrays" that are in many ways equivalent to Numpy ndarrays except that they live in the GPU's global memory, rather than on the CPU. These device arrays can be passed to Numba cuda functions just the way Numpy arrays can, but without the memory copying overhead.

We can also create an output buffer on the GPU with the numba.cuda.device_array() function:

out_device = cuda.device_array(shape=(n,), dtype=np.float32)

Now we can try out our Cuda kernel using the pre-copied device arrays, and compare the time to a version without moving the data first.

%timeit add_kernel[blocks_per_grid, threads_per_block](x_device, y_device, out_device)
%timeit add_kernel[blocks_per_grid, threads_per_block](x, y, out)

```
The slowest run took 6.66 times longer than the fastest. This could mean that an intermediate result is being cached.
1000 loops, best of 5: 291 µs per loop
100 loops, best of 5: 2.96 ms per loop
```

As you can see, moving data back and forth is expensive. In general, you will want to keep your data on the GPU as long as possible and do as many calculations as you can before moving it back. Of course, at some point you will have to move the data back to the CPU, whether to output to a file, perform some other CPU functions, etc.. Numba provides the copy_to_host function for that:

out = out_device.copy_to_host()

After all of this extra work, you may be wondering what the point of doing calculations on the GPU is. After all, if we compare the time it takes to run our fancy Cuda kernel with the pre-allocated GPU arrays, to the CPU version provided by python, we find that the CPU version is still faster:

%timeit x + y

But that's because we still haven't given the GPU enough work to do. If we go back and try the same problem, but with arrays that are 10 times larger.

n = 1000000
x = np.arange(n).astype(np.float32)
y = 2 * x
x_device = cuda.to_device(x)
y_device = cuda.to_device(y)
out_device = cuda.device_array(shape=(n,), dtype=np.float32)
%timeit add_kernel[blocks_per_grid, threads_per_block](x_device, y_device, out_device)

```
The slowest run took 5.57 times longer than the fastest. This could mean that an intermediate result is being cached.
1000 loops, best of 5: 313 µs per loop
```

%timeit x + y

```
The slowest run took 6.22 times longer than the fastest. This could mean that an intermediate result is being cached.
1000 loops, best of 5: 742 µs per loop
```

we start to see the power of the GPU. As you may have noticed, the GPU function called with a million array elements took approximately the same amount of time to run as the version with 100 thousand elements. That's because most of the GPU was not busy when we only had 100 thousand elements, so we still hadn't overcome the overhead of launching the kernel.